

标注“额外内容”的是课上没提到的。

INT102只需要用伪代码实现。

访问这个网站（非常多算法）：<https://oi-wiki.org/>

[1] 搜索算法

线性搜索

```
1 public static int linearSearch(int[] array, int number)
2 {
3     for(int i =0;i<array.length;i++)
4         if(array[i]==number)
5             return i;          // 找到了, 返回index
6     return -1;                // 没找到index, 返回-1
7 }
```

二分搜索

递归 Recursive

```
1 public static int binarySearch(int[] array, int number){
2     if(array == null || array.length==0) return -1; // 如果数组是空的, 返回-1
3     else return binarySearch(array,number,0,array.length-1); // 开始二分搜索
4 }
5 private static int binarySearch(int[] array,int number, int left, int right){
6     if(left>right) return -1; // 如果左边大于右边, 直接返回-1。因为越界代表找不到
7     int middleValue = array[(left+right)/2]; // 定义中间值
8     if(number == middleValue) return (left+right)/2; // 如果相等就返回
9     if(number < middleValue) return binarySearch(array,number,left,(left+right)/2-1);
10    // 如果number比中间值小, 就往左边找
11    return binarySearch(array,number,(left+right)/2+1,right); // 如果number比中间值大, 就
    往右边找
11 }
```

循环方法

```
1 public static int binarySearchLoop(int[] array, int number)
2 {
3     if(array == null || array.length==0) return -1; // 如果数组是空的, 返回-1
4     int left = 0;
5     int right = array.length-1;
6     while(left<=right)
7     {
8         int middleValue = array[(left+right)/2];
9         if(number > middleValue){ // 往右边找
10             left = (left+right)/2 +1;
11             continue;
12         }
13         if(number < middleValue){ // 往左边找
14             right = (left+right)/2-1;
15             continue;
16         }
17         if(number == middleValue)
18             return (left+right)/2;
```

```
19     }
20     return -1;
21 }
```

时间复杂度 $O(nm)$

给定一个文本 `text` (String型) 和一个模式 `pattern`。要求找到这个文本 `text` 内是否含有 `pattern`。

```
1 public static boolean exist(String text, String pattern)
2 {
3     if(text==null||pattern==null|| text.isEmpty() || pattern.isEmpty()) return false;
4     if(pattern.length()>text.length()) return false;
5
6
7     for(int i = 0;i<text.length();i++)                // 第一层循环
8     {
9         for(int j = 0; j<pattern.length();j++)        // 第二层循环
10        {
11            if(pattern.charAt(j)!=text.charAt(j+i)) break;
12            if(j==pattern.length()-1) return true;
13        }
14    }
15    return false;
16 }
```

时间复杂度 $O(n)$

[看这个视频!](#)

```

1 public class Horspool {
2     public static boolean contains(String text, String pattern)
3     {
4         return getIndex(text, pattern) != -1;
5     }
6     public static int getIndex(String text, String pattern) {
7         if(text == null || pattern == null) return -1;
8         if(text.length() < pattern.length()) return -1;
9
10        HashMap<Character, Integer> map = getBadMatchTable(pattern);
11        int index = pattern.length()-1;
12        while(index <= text.length()-1)
13        {
14            boolean equals = true;
15            int patternPointer = pattern.length()-1;
16            int textPointer = index;
17            while(patternPointer >= 0 && equals)
18                if(text.charAt(textPointer) == pattern.charAt(patternPointer))
19                    {textPointer--;patternPointer--;}

```

```

20         else
21             equals = false;
22
23         if(equals)
24             return index - pattern.length() + 1;
25         else
26             index += map.getOrDefault(text.charAt(index), pattern.length());
27     }
28     return -1;
29 }
30 private static HashMap<Character,Integer> getBadMatchTable(String pattern) {
31     HashMap<Character,Integer> map = new HashMap<>();
32     int lengthOfPattern = pattern.length();
33     for(int index = 0; index < lengthOfPattern-1; index++)
34         map.put(pattern.charAt(index), lengthOfPattern - index - 1);
35     map.putIfAbsent(pattern.charAt(lengthOfPattern-1), lengthOfPattern);
36     return map;
37 }
38 }
39

```

KMP

看[这个视频](#) 和 [这个文档](#)

```

1  public class KMP {
2      public static void main(String[] args) {
3          System.out.println(getIndex("mississippi","issip"));
4      }
5      // 获取首个index
6      public static int getIndex(String text, String pattern) {
7          text = STR."\{pattern}#\{text}";
8          int[] PiArray = new int[text.length()];
9          for (int i = 1; i < text.length(); i++) {
10             int index = PiArray[i-1];
11             while(index != 0 && text.charAt(i) != text.charAt(index))
12                 index = PiArray[index-1];
13             if(text.charAt(index) == text.charAt(i))
14                 index++;
15             PiArray[i] = index;
16             if(index == pattern.length())
17                 return i - 2 * pattern.length();
18         }
19         return -1;
20     }
21     // 获取所有 index
22     public static ArrayList<Integer> getAllIndex(String text, String pattern) {
23         ArrayList<Integer> list = new ArrayList<Integer>();
24         if(pattern == null || text == null || pattern.isEmpty() || text.isEmpty())
25             return new ArrayList<>();
26         if(pattern.length() > text.length())
27             return new ArrayList<>();

```

```
28     int[] PiArray = getPIArray(pattern+'#'+text);
29     for (int i = 0; i < PiArray.length; i++) {
30         if (PiArray[i] == pattern.length())
31             list.add( i - 2 * pattern.length());
32     }
33     return list;
34 }
35 private static int[] getPIArray(String text) {
36     int[] PiArray = new int[text.length()];
37     for (int i = 1; i < text.length(); i++) {
38         int index = PiArray[i-1];
39         while(index != 0 && text.charAt(i) != text.charAt(index))
40             index = PiArray[index-1];
41         if(text.charAt(index) == text.charAt(i))
42             index++;
43         PiArray[i] = index;
44     }
45     return PiArray;
46 }
47 }
48
```

[2] 排序算法

选择排序

[看这个视频!](#)

```
1 public static void selectionSort(int[] array){
2     if(array == null || array.length==0) return;
3     for(int i = 0; i<array.length-1;i++)
4     {
5         for(int j=i+1;j<array.length;j++)
6         {
7             if(array[j]<array[i])
8                 swap(array,i,j);
9         }
10    }
11 }
12 private static void swap(int[] array, int indexA, int indexB)
13 {
14     int temp = array[indexA];
15     array[indexA] = array[indexB];
16     array[indexB] = temp;
17 }
```

冒泡排序

[看这个视频!](#)

```
1 public static void bubbleSort(int[] array){
2     if(array == null || array.length==0) return;
3     for(int end = array.length-1;end>=0;end--)
4     {
5         for(int begin = 0;begin<end;begin++)
6         {
7             if(array[begin]>array[begin+1]) swap(array,begin,begin+1);
8         }
9     }
10 }
```

插入排序

[看这个视频!](#)

```

1 public static void insertSort(int[] array){
2     if(array == null || array.length==0) return;
3     for(int i = 1;i<array.length;i++)
4     {
5         int j = i-1;
6         while(j>=0 && array[j]>array[j+1])
7         {
8             swap(array,j,j+1);
9             j--;
10        }
11    }
12 }

```

归并排序

[看这个视频!](#)

```

1 public static void mergeSort(int[] arr) {
2     if (arr == null || arr.length < 2) return;
3     mergeSort(arr, 0, arr.length - 1);
4 }
5
6 private static void mergeSort(int[] array, int left, int right) {
7     // 这里左右都要分，分到最小单元之后merge。
8     if (left < right) {
9         int middle = left + (right - left) / 2;
10        mergeSort(array, left, middle);
11        mergeSort(array, middle + 1, right);
12        merge(array, left, middle, right);
13    }
14 }
15
16 private static void merge(int[] array, int left, int middle, int right) {
17     // 第一个数组: left 到 middle
18     // 第二个数组: middle+1 到 right
19     // 现在选出两个数组最小的放在temp中
20     int[] temp = new int[right - left + 1];
21     int indexOfTemp = -1;
22     int firstArrayIndex = left ;
23     int secondArrayIndex = middle+1;
24     while (firstArrayIndex <= middle && secondArrayIndex <= right)
25         if (array[firstArrayIndex] < array[secondArrayIndex])
26             temp[++indexOfTemp] = array[firstArrayIndex++];
27         else
28             temp[++indexOfTemp] = array[secondArrayIndex++];
29
30     // 复制剩余数组到 temp
31     if (firstArrayIndex == middle+1) // 如果first到末尾了
32         while (secondArrayIndex <= right) temp[++indexOfTemp] =
array[secondArrayIndex++];
33     else while (firstArrayIndex <= middle) temp[++indexOfTemp] =
array[firstArrayIndex++];

```

```

34
35     // temp替换原来数组
36     indexOfTemp = -1;
37     while (++indexOfTemp < temp.length)
38         array[left + indexOfTemp] = temp[indexOfTemp];
39 }
40
41 public static void main(String[] args) {
42     int[] arr = {51, 13, 10, 64, 34, 5, 32, 21};
43     mergeSort(arr);
44     System.out.println(Arrays.toString(arr));
45 }

```

快速排序

```

1  import java.util.Arrays;
2  import java.util.Collections;
3  import java.util.Random;
4
5  public class QuickSort {
6      private static final Random rand = new Random(System.currentTimeMillis());
7      public static void sort(int[] arr) {
8          if(arr == null || arr.length < 2) {
9              return;
10         }
11         sort(arr, 0, arr.length - 1);
12     }
13     private static void sort(int[] arr, int left, int right) {
14         if(left >= right) return;
15         int pivot = getPivot(arr, left, right);
16         sort(arr, left, pivot - 1);
17         sort(arr, pivot + 1, right);
18     }
19     private static int getPivot(int[] arr, int left, int right) {
20         int i = left;
21         int j = right;
22         // 随机一个作为pivot, 并移动到左边
23         swap(arr, left, rand.nextInt(right - left + 1) + left);
24         int pivot = arr[left];
25         while(i < j){
26             while(i < j && arr[j] > pivot){j--;}
27             while(i < j && arr[i] <= pivot){i++;}
28             swap(arr, i, j);
29         }
30         // i == j now
31         swap(arr, i, left);
32         return i;
33     }
34     private static void swap(int[] arr, int i, int j) {
35         int temp = arr[i];
36         arr[i] = arr[j];
37         arr[j] = temp;

```

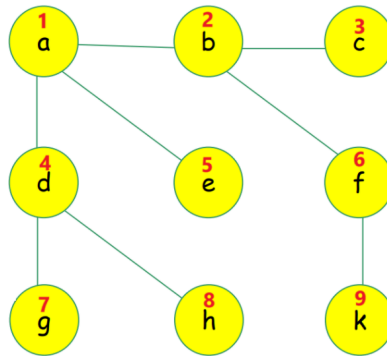


```
38     }
39     public static void main(String[] args) {
40         int[] arr = {5,1,1,2,0,0};
41         Arrays.parallelSort(arr);
42         QuickSort.sort(arr);
43         for (int num : arr) {
44             System.out.print(num + " ");
45         }
46     }
47 }
48
```

[3] 实现BFS和DFS

这个可以看课件

假设我们有如下图



创建图

```
1 public class Graph {
2     private boolean[][] graph; // 这个数组是邻接矩阵, graph[a][b]代表ab点是否联通
3     private boolean[] flag; // 这个数组代表这个点是否被标记
4     private int size;
5     public Graph(int size) {
6         this.size = size+1;
7         createGraph();
8     }
9     private void createGraph() {
10        graph = new boolean[size][size];
11        flag = new boolean[size];
12    }
13    public void connect(int pointA, int pointB) { // 链接两个点
14        graph[pointA][pointB] = true;
15        graph[pointB][pointA] = true;
16    }
17    public boolean isConnected(int pointA, int pointB) { // 检查两个点是否链接
18        return graph[pointA][pointB];
19    }
20    public ArrayList<Integer> getAdjacentPoints(int pointA) { // 返回A的所有邻接点
21        ArrayList<Integer> adjacentPoints = new ArrayList<>();
22        for (int i = 0; i < size; i++)
23            if (graph[pointA][i])
24                adjacentPoints.add(i);
25        return adjacentPoints;
26    }
27    public void flagThisPoint(int pointA) { // 标记这个点
28        flag[pointA] = true;
29    }
30    public void unFlagThisPoint(int pointA) { // 取消标记这个点
```

```

31     flag[pointA] = false;
32 }
33 public boolean isFlagged(int pointA) { // 返回boolean: 这个点是否被标记
34     return flag[pointA];
35 }
36 public ArrayList<Integer> getUnFlaggedPoints(int point) {
37     ArrayList<Integer> unFlaggedPoints = new ArrayList<>();
38     for (int i = 0; i < size; i++)
39     {
40         if(isConnected(point, i)&& !isFlagged(i))
41             unFlaggedPoints.add(i);
42     }
43     return unFlaggedPoints;
44 }
45 }

```

BFS

```

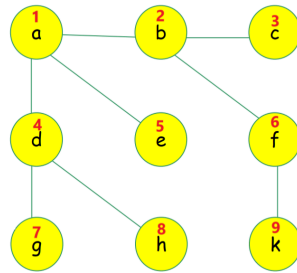
1 public class BFS {
2     public static void main(String[] args) {
3         // 创建大小为9个点的图
4         Graph graph = new Graph(9);
5         // 连接点
6         graph.connect(1,2);
7         graph.connect(2,3);
8         graph.connect(2,6);
9         graph.connect(6,9);
10        graph.connect(1,5);
11        graph.connect(1,4);
12        graph.connect(4,8);
13        graph.connect(4,7);
14
15        ArrayList<Integer> bfs = new ArrayList<>(); // 记录BFS的路径
16        ArrayList<Integer> temp = new ArrayList<>(); // 记录待遍历点, 相当于缓存
17        temp.add(1);
18        graph.flagThisPoint(0); // 标记待遍历点
19        while(!temp.isEmpty())
20        {
21            int vertex = temp.removeFirst(); // 删除first点, 同时赋值到vertex中
22            if(!graph.isFlagged(vertex)) // 如果这个点没有被标记过
23            {
24                bfs.add(vertex);
25                graph.flagThisPoint(vertex);
26            }
27            temp.addAll(graph.getUnFlaggedPoints(vertex));
28        }
29        System.out.println(Arrays.toString(bfs.toArray()));
30    }
31 }

```

输入如下:

```
1 | [1, 2, 4, 5, 3, 6, 7, 8, 9]
```

正好是广度优先搜索的顺序



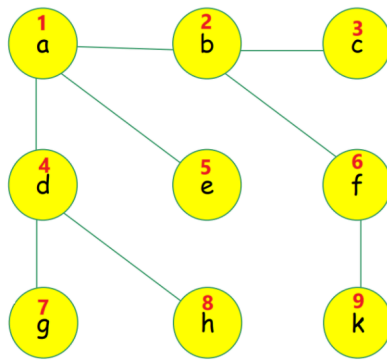
DFS

```
1 public class DFS{
2     // 创建大小为9个点的图
3     private static Graph graph = new Graph(9);
4     private static ArrayList<Integer> path = new ArrayList();
5     public static void main(String[] args) {
6         graph.connect(1,2);
7         graph.connect(2,3);
8         graph.connect(2,6);
9         graph.connect(6,9);
10        graph.connect(1,5);
11        graph.connect(1,4);
12        graph.connect(4,8);
13        graph.connect(4,7);
14        path.add(1); // 添加待遍历点
15        graph.flagThisPoint(1); // 标记待遍历点
16        dfs(1); // 开始深搜
17        System.out.println(path);
18    }
19
20    private static void dfs(int vertex) {
21        graph.getUnFlaggedPoints(vertex).forEach(point ->{
22            if(!graph.isFlagged(point)){
23                path.add(point);
24                graph.flagThisPoint(point);
25                dfs(point);
26            }
27        });
28    }
29 }
```

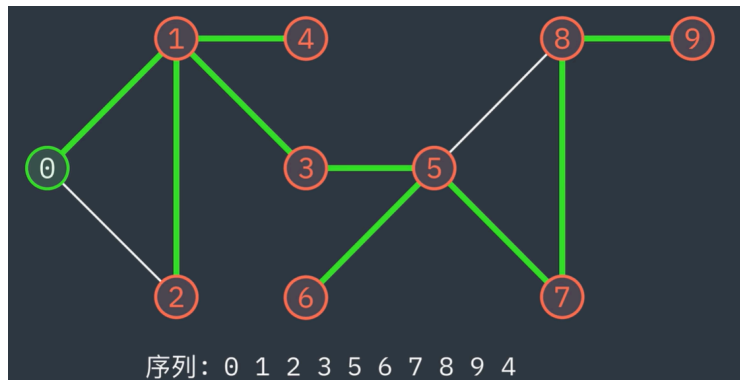
输出

```
1 | [1, 2, 3, 6, 9, 4, 7, 8, 5]
```

确实是深度搜索：



当然换成这个例子，代码也是正确的输出

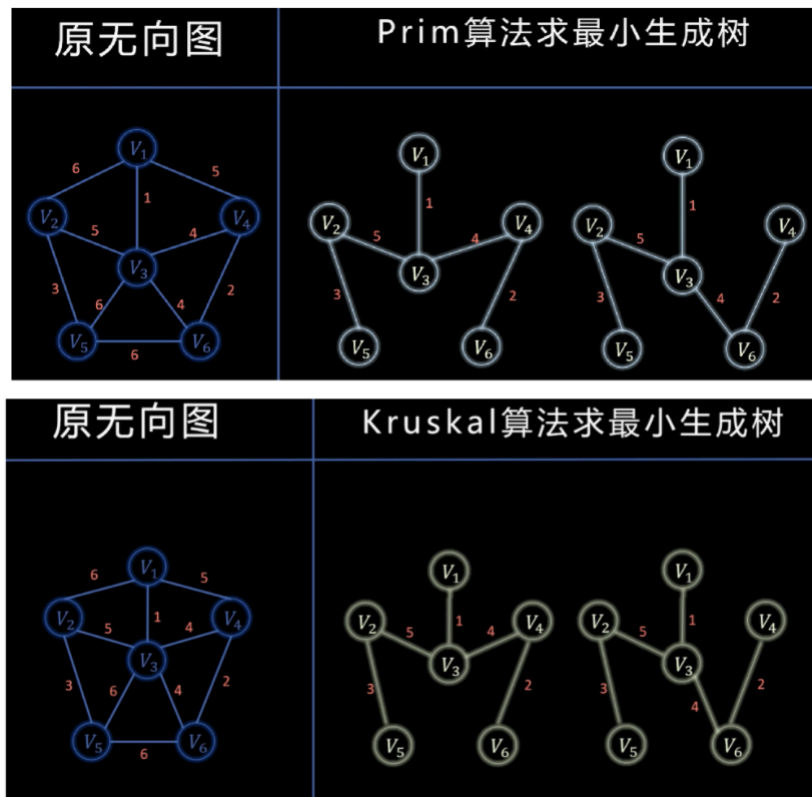


[4] 图论

下面的实现都是基于MyMap类的，这个类会放到最后

最小生成树之 Prim 和 Kruskal 算法

[看这个视频!](#)



首先，我们自己创建一个 `MyMap` 类，这个类包含了图的一些基本的功能...在 `Main` 函数下，我们可以这样写

```
1 public static void main(String[] args) {
2     MyMap map = new MyMap(MyMap.UNDIRECTED_GRAPH); // 无向图
3     // 连接点
4     map.connect("v1", "v2", 6);
5     map.connect("v1", "v3", 1);
6     map.connect("v1", "v4", 5);
7     map.connect("v2", "v3", 5);
8     map.connect("v2", "v5", 3);
9     map.connect("v3", "v4", 4);
10    map.connect("v3", "v5", 6);
11    map.connect("v3", "v6", 4);
12    map.connect("v4", "v6", 2);
13    map.connect("v5", "v6", 6);
14    map.Prim();
15    System.out.println("-----");
16    map.Kruskal();
17 }
```

输出

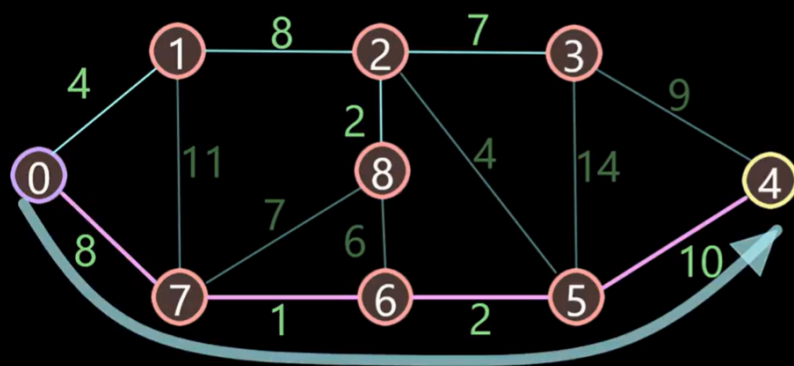
```
1 V6<->V4=2
2 V6<->V3=4
3 V3<->V1=1
4 V3<->V2=5
5 V2<->V5=3
6 -----
7 V3<->V1=1
8 V6<->V4=2
9 V5<->V2=3
10 V6<->V3=4
11 V2<->V3=5
```

Dijkstra 单源最短路径算法

无向图的 Dijkstra

[看这个视频!](#)

1. 每次从未标记的节点中选择距离出发点最近的节点，标记，收录到最优路径集合中。
2. 计算刚加入节点A的邻近节点B的距离（不包含标记的节点），若（节点A的距离+节点A到节点B的边长）< 节点B的距离，就更新节点B的距离和前面点。



	出发点0	前面点
0	0	
1	4	0
2	12	1
3	19	2
4	21	5
5	11	6
6	9	7
7	8	0
8	14	2

```
1 public static void main(String[] args) {
2     MyMap map = new MyMap(MyMap.UNDIRECTED_GRAPH); //无向图的 Dijkstra
3     map.connect("0", "1", 4);
4     map.connect("1", "2", 8);
5     map.connect("2", "3", 7);
6     map.connect("3", "4", 9);
7     map.connect("4", "5", 10);
8     map.connect("5", "6", 2);
9     map.connect("6", "7", 1);
10    map.connect("7", "0", 8);
11
12    map.connect("1", "7", 11);
13    map.connect("2", "8", 2);
```

```

14 map.connect("8", "6", 6);
15 map.connect("7", "8", 7);
16 map.connect("2", "5", 4);
17 map.connect("3", "5", 14);
18
19 map.Dijkstra("0", "4");
20 }

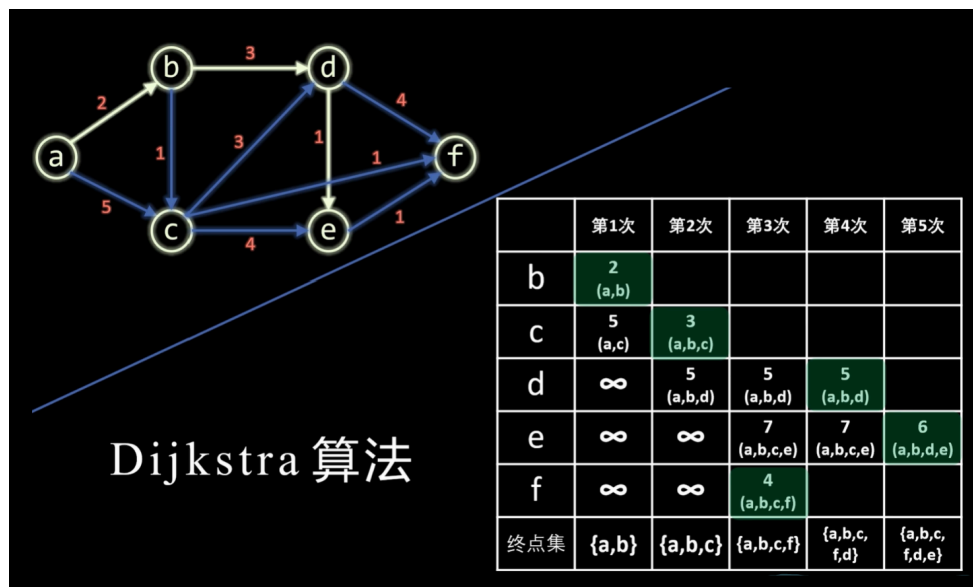
```

输出

```
1 | ["4"]<---21--->["5"]<---11--->["6"]<---9--->["7"]<---8--->["0"]<---0--->
```

有向图的 Dijkstra

[看这个视频!](#)



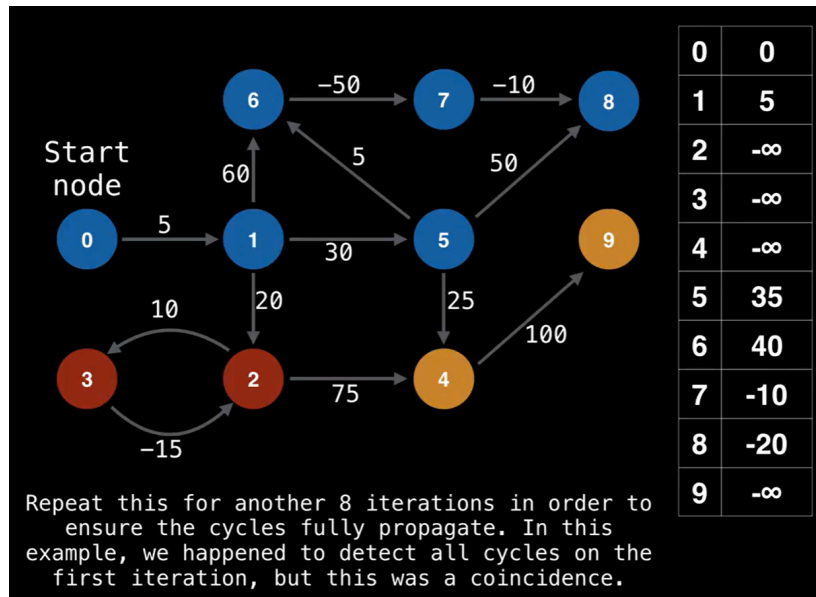
```

1 public static void main(String[] args) {
2     MyMap map = new MyMap(MyMap.DIRECTED_GRAPH);
3     map.connect("a", "b", 2);
4     map.connect("a", "c", 5);
5     map.connect("b", "c", 1);
6     map.connect("b", "d", 3);
7     map.connect("c", "d", 3);
8     map.connect("c", "e", 4);
9     map.connect("c", "f", 1);
10    map.connect("d", "e", 1);
11    map.connect("d", "f", 4);
12    map.connect("e", "f", 1);
13    map.Dijkstra("a", "f");
14 }

```

Bellman-Ford算法

看 [这个视频!](#) 和 [这个视频!](#)



```

1 public static void main(String[] args) {
2     MyMap map = new MyMap(MyMap.DIRECTED_GRAPH);
3     map.connect ("0","1",5);
4     map.connect ("1","2",20);
5     map.connect ("1","5",30);
6     map.connect ("1","6",60);
7     map.connect ("2","3",10);
8     map.connect ("3","2",-15);
9     map.connect ("2","4",75);
10    map.connect ("4","9",100);
11    map.connect ("5","4",25);
12    map.connect ("5","6",5);
13    map.connect ("5","8",50);
14    map.connect ("6","7",-50);
15    map.connect ("7","8",-10);
16    map.BellmanFord("0","9");
17 }

```

输出

```

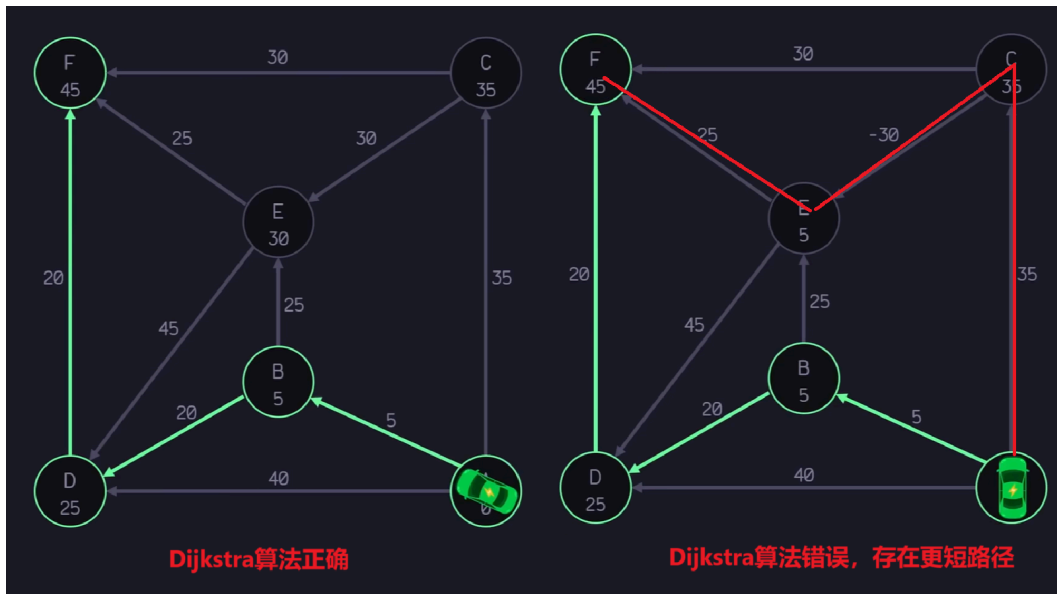
1 {0=0, 1=5, 2=-2147483647, 3=-2147483647, 4=-2147483647, 5=35, 6=40, 7=-10, 8=-20,
  9=-2147483647}

```

Floyd 算法

观看 [这个视频!](#)

主要看那个distance表如何更新，绕行顶点表如何记录（不用全看完）



这张图如果把那条 CE 权重换位-30，Dijkstra算法无法解决，因此可以使用Floyd算法

```
1 public static void main(String[] args) {
2     MyMap map = new MyMap(MyMap.DIRECTED_GRAPH);
3     map.connect("A", "B", 5);
4     map.connect("A", "C", 35);
5     map.connect("A", "D", 40);
6     map.connect("B", "D", 20);
7     map.connect("B", "E", 25);
8     map.connect("E", "D", 45);
9     map.connect("C", "E", 30); // 改为-30 则为 F<--E<--C<--A
10    map.connect("C", "F", 30);
11    map.connect("E", "F", 25);
12    map.connect("D", "F", 20);
13    map.Floyd("A", "F"); // 否则为 F<--D<--B<--A
14 }
```

输出

```
1 F<--D<--B<--A
```

Warshall's 算法

[看这个视频!](#)

```
1 public static void main(String[] args) {
2     MyMap map = new MyMap(MyMap.DIRECTED_GRAPH);
3     map.connect("A", "B", 5);
4     map.connect("A", "C", 35);
5     map.connect("A", "D", 40);
6     map.connect("B", "D", 20);
7     map.connect("B", "E", 25);
8     map.connect("E", "D", 45);
9     map.connect("C", "E", 30);
```

```

10     map.connect("C", "F", 30);
11     map.connect("E", "F", 25);
12     map.connect("D", "F", 20);
13     System.out.println(map.warshall("A", "F"));
14 }

```

和Folyd用图一样

输出

```
1 | true
```

代表 A 和 F 能连接

MyMap 类

```

1  import java.util.*;
2
3
4  public class MyMap {
5      public static final int DIRECTED_GRAPH = 1; // 有向图
6      public static final int UNDIRECTED_GRAPH = 2; // 无向图
7      private final HashMap<Node, Integer> map = new HashMap<>(); // 整个图，记录了点和权重
8      private final HashSet<HashSet<String>> set = new HashSet<>(); // [无向图]分组
9      private final HashMap<String, LinkedHashSet<String>> adjList = new HashMap<>(); //
    [有向图]邻接表，参数1是头，参数2是邻接表
10     private final HashMap<String, Boolean> nodes = new HashMap<>(); // 表示点，以及有没有
    被标记
11     private final HashMap<Node, Boolean> edge = new HashMap<>(); // 表示边，以及边是否被
    标记
12     private final int MODE;
13     private int totalNodesNumber;
14     private int flaggedNodesNumber;
15     private int totalEdgesNumber;
16     private int flaggedEdgesNumber;
17
18     public MyMap(int GRAPH_MODE) {
19         this.MODE = GRAPH_MODE;
20         if (MODE != DIRECTED_GRAPH && MODE != UNDIRECTED_GRAPH) {
21             throw new IllegalArgumentException("必须是有向图或者无向图之一");
22         }
23     } // 创建的时候必须指定图的类型
24     public boolean connect(Object a, Object b, int weight){
25         return connect(a.toString(), b.toString(), weight);
26     }
27     public boolean connect(String a, String b, int weight) {
28         switch (MODE) {
29             case DIRECTED_GRAPH -> {
30                 return connectDirectedGraph(a, b, weight);
31             }
32             case UNDIRECTED_GRAPH -> {

```

```

33         return connectUndirectedGraph(a, b, weight);
34     }
35 }
36 return false;
37 }
38
39 private boolean connectDirectedGraph(String a, String b, int weight) {
40     // 如果这个图已经记录了ab直接连通，那么返回，连接失败
41     if (map.containsKey(new Node(a, b))) return false;
42
43     // 连接这两个点，现在他们连通了
44     map.put(new Node(a, b), weight);
45
46     // 邻接表
47     // 先检查邻接表有没有这个点：
48     // 如果存在，直接把B添加到邻接表后头
49     // 否则，就创建点A，然后添加点B
50     boolean containsNodeA = adjList.containsKey(a);
51     if (containsNodeA) {
52         adjList.get(a).add(b);
53     } else {
54         LinkedHashSet<String> nodes = new LinkedHashSet<>();
55         nodes.add(a);
56         nodes.add(b);
57         adjList.put(a, nodes);
58     }
59     // 记录点数量
60     if (!nodes.containsKey(a)) ++totalNodesNumber;
61     if (!nodes.containsKey(b)) ++totalNodesNumber;
62     // 添加点 node
63     nodes.put(a, false);
64     nodes.put(b, false);
65     // 添加边 edge
66     edge.put(new Node(a, b), false);
67     // 记录边数量 totalEdgeNumber
68     totalEdgesNumber++;
69
70     return true;
71 } // 有向图
72
73 private boolean connectUndirectedGraph(String a, String b, int weight) {
74
75     // 如果这个图已经记录了ab直接连通，那么返回，连接失败
76     if (map.containsKey(new Node(a, b)) || map.containsKey(new Node(b, a))) return
false;
77
78     // 连接这两个点
79     map.put(new Node(a, b), weight);
80     map.put(new Node(b, a), weight);
81
82     // 给他们划分组别，如果连通就放在一组，否则就新开一个组
83     // 有一些情况：
84     // 点A和B都被记录，而且都在同一个组：不需要分组

```

```

85 // 点A和B都被记录，但在同一个组：合并他们的组
86 // 点A和B其中一个被记录：直接添加另一个到当前组
87 // 点A和B都没有被记录：创建新组
88 boolean containsNodeA = nodes.containsKey(a);
89 boolean containsNodeB = nodes.containsKey(b);
90 HashSet<String> setA = new HashSet<>();
91 HashSet<String> setB = new HashSet<>();
92 boolean setAFound = false;
93 boolean setBFound = false;
94 boolean inSameSet = false;
95 // 点A和B都被记录
96 if (containsNodeA && containsNodeB) {
97     Iterator<HashSet<String>> iterator = set.iterator();
98     while (iterator.hasNext()) {
99         HashSet<String> sets = iterator.next();
100         boolean setContainsA = sets.contains(a);
101         boolean setContainsB = sets.contains(b);
102         // 而且都在同一个组：不需要分组
103         if (setContainsA && setContainsB) break;
104         // 但在同一个组：合并他们的组
105         if (setContainsA) {
106             setA = sets;
107             setAFound = true;
108         }
109         if (setContainsB) {
110             setB = sets;
111             setBFound = true;
112         }
113         if (setAFound && setContainsB) {
114             setA.addAll(sets);
115             iterator.remove();
116             break;
117         }
118         if (setBFound && setContainsA) {
119             setB.addAll(sets);
120             iterator.remove();
121             break;
122         }
123     }
124 }
125 // 点A和B其中一个被记录
126 if (containsNodeA || containsNodeB) {
127     // 直接添加另一个到当前组
128     for (HashSet<String> sets : set) {
129         if (sets.contains(a)) {
130             sets.add(b);
131             break;
132         }
133         if (sets.contains(b)) {
134             sets.add(a);
135             break;
136         }
137     }

```

```

138     }
139     // 点A和B都没有被记录：创建新组
140     if (!containsNodeA && !containsNodeB) {
141         HashSet<String> newHashSet = new HashSet<>();
142         newHashSet.add(a);
143         newHashSet.add(b);
144         set.add(newHashSet);
145     }
146
147     // 记录点数量
148     if (!containsNodeA) totalNodesNumber++;
149     if (!containsNodeB) totalNodesNumber++;
150
151
152     // 添加点
153     nodes.put(a, false);
154     nodes.put(b, false);
155
156     // 添加边
157     edge.put(new Node(a, b), false);
158     edge.put(new Node(b, a), false);
159
160     // 记录边数量
161     if (!(containsNodeA && containsNodeB)) totalEdgesNumber++;
162
163     return true;
164 } // [无向图]链接这两个点，以及他们的权重，返回一个boolean表示他们是否添加成功
165
166 public boolean isConnected(Node node) {
167
168     switch (MODE) {
169         case DIRECTED_GRAPH -> {
170             // 如果其中一个点没有被记录过，自然也不是连接的
171             if (!nodes.containsKey(node.A) || !nodes.containsKey(node.B)) return
false;
172
173             // 如果这两个点是直接连接，就直接返回true
174             if (isDirectConnected(node.A, node.B)) return true;
175             // 现在，要判断这两个点是否间接连接
176             // 遍历邻接表，从a开始，一直找，看看能否找到b
177             // 遍历过的点需要标记
178             HashSet<String> visited = new HashSet<>();
179             LinkedHashSet<String> nodeShouldCheckItsAdjList = new LinkedHashSet<>
();
180
181             LinkedHashSet<String> adjListOfA = adjList.get(node.A);
182             // 一直找点a和b是否连接
183             while (visited.size() != getTotalNodeNumber()) {
184                 // 遍历a点的邻接表
185                 for (String nodes : adjListOfA) {
186                     // 如果当前点nodes和目标点b相等就返回 true
187                     if (nodes.equals(node.B)) return true;
188                     // 否则，把这个nodes标记为已访问，然后加入
189                     nodeShouldCheckItsAdjList
// 意思是下一次需要访问这个点的邻接表

```

```

188         else {
189             if (nodes.equals(node.A)) visited.add(nodes); // 如果这个表
是表头，就不再访问
190
191             // 如果nodes有邻接表而且这个点还没有被访问
192             if (adjList.containsKey(nodes) &&
!visited.contains(nodes))
193                 nodeShouldCheckItsAdjList.add(nodes);
194         }
195     }
196     adjListOfA = adjList.get(nodeShouldCheckItsAdjList.removeLast());
197 }
198 return false;
199 }
200 case UNDIRECTED_GRAPH -> {
201     // 如果其中一个点没有被记录过，自然也不是连接的
202     if (!nodes.containsKey(node.A) || !nodes.containsKey(node.B)) return
false;
203
204     // 如果这两个点是直接连接，就直接返回true
205     if (isDirectConnected(node.A, node.B)) return true;
206     // 现在，要判断这两个点是否间接连接（也就是检查是不是在一个组）
207     for (HashSet<String> sets : set) {
208         if (sets.contains(node.A)) return sets.contains(node.B);
209         if (sets.contains(node.B)) return sets.contains(node.A);
210     }
211 }
212
213 return false;
214 }
215
216 public boolean isConnected(String a, String b) {
217     return isConnected(new Node(a, b));
218 } // 检查这两个点是不是直接/间接连接的
219
220 public boolean isDirectConnected(String a, String b) {
221     return map.containsKey(new Node(a, b));
222 } // 检查是否直接连接的
223
224 public void unFlagAllPoint() {
225     nodes.forEach((a, b) -> {
226         b = false;
227     });
228     flaggedNodesNumber = 0;
229 } // 取消标记所有点
230
231
232 public void flagAllPoint() {
233     nodes.forEach((a, b) -> {
234         b = true;
235     });
236     flaggedNodesNumber = totalNodesNumber;
237 } // 标记所有点

```

```
238
239
240 public void flagPoint(String point) {
241     if (nodes.containsKey(point)) {
242         nodes.put(point, true);
243         flaggedNodesNumber++;
244     }
245 } // 标记某个点
246
247
248 public void unFlagPoint(String point) {
249     if (nodes.containsKey(point)) {
250         nodes.put(point, false);
251         flaggedNodesNumber--;
252     }
253 } // 取消标记某个点
254
255 public void flagAllEdge() {
256     edge.forEach((a, b) -> {
257         b = true;
258     });
259 } // 标记所有边
260
261 public void unFlagAllEdge() {
262     edge.forEach((a, b) -> {
263         b = false;
264     });
265 } // 取消标记所有边
266
267 public void flagEdge(String pointA, String pointB) {
268     if (map.containsKey(new Node(pointA, pointB))) {
269         edge.put(new Node(pointA, pointB), true);
270         flaggedEdgesNumber++;
271     }
272 } // 标记某条边
273
274 public void unFlagEdge(String pointA, String pointB) {
275     if (map.containsKey(new Node(pointA, pointB))) {
276         edge.put(new Node(pointA, pointB), false);
277         flaggedEdgesNumber--;
278     }
279 } // 取消标记某条边
280
281 public boolean isAllFlagged() {
282     return flaggedNodesNumber == totalNodesNumber;
283 } // 是否全部点都标记了
284
285
286 public boolean isAllUnFlagged() {
287     return flaggedNodesNumber == 0;
288 } // 是否全部点都没标记
289
290 /**
```



```

291     * 返回这个点所有的直接连接点，以及对应的权重
292     * 如果是无向图，返回所有连接点
293     * 如果是有向图，返回邻接表
294     */
295     public HashMap<String, Integer> getAllNearbyPoint(String point) {
296         // 例如getAllNearbyPoint("A")
297         // 返回一个HashMap，包含<String, Integer> 第一个参数是点名称，第二个是权重
298         HashMap<String, Integer> returnMap = new HashMap<>();
299         nodes.forEach((a, b) -> {
300             if (map.containsKey(new Node(point, a))) returnMap.put(a, map.get(new
Node(point, a)));
301         });
302         return returnMap;
303     }
304
305     public HashSet<String> getAllUnFlaggedPoint() {
306         HashSet<String> returnSet = new HashSet<>();
307         nodes.forEach((a, b) -> {
308             if (!b) returnSet.add(a);
309         });
310         return returnSet;
311     } // 返回所有未标记点
312
313     public HashSet<String> getAllFlaggedPoint() {
314         HashSet<String> returnSet = new HashSet<>();
315         nodes.forEach((a, b) -> {
316             if (b) returnSet.add(a);
317         });
318         return returnSet;
319     } // 返回所有标记点
320
321     public HashMap<Node, Integer> getAllUnFlaggedEdge() {
322         HashMap<Node, Integer> returnSet = new HashMap<>();
323         edge.forEach((a, b) -> {
324             if (!b) returnSet.put(a, map.get(a));
325         });
326         return returnSet;
327     } // 返回所有未标记边
328
329     public HashMap<Node, Integer> getAllFlaggedEdge() {
330         HashMap<Node, Integer> returnSet = new HashMap<>();
331         edge.forEach((a, b) -> {
332             if (b) returnSet.put(a, map.get(a));
333         });
334         return returnSet;
335     } // 返回所有标记边
336
337     public boolean isEmpty() {
338         return nodes.isEmpty();
339     } // map是否为空
340
341     public String getClosestPointName(String objectPoint) {
342         if (!nodes.containsKey(objectPoint)) return null;

```

```

343
344     String returnPoint = null;
345     int weight = Integer.MAX_VALUE;
346     for (String nodes : nodes.keySet()) {
347         if (map.containsKey(new Node(objectPoint, nodes)) && weight > map.get(new
Node(objectPoint, nodes))) {
348             weight = map.get(new Node(objectPoint, nodes));
349             returnPoint = nodes;
350         }
351     }
352     return returnPoint;
353 } // 返回这个点周围与他最小权重的点的名字
354
355
356 public int getClosestPointWeight(String objectPoint) {
357     if (!nodes.containsKey(objectPoint)) return 0;
358
359     String returnPoint = null;
360     int weight = Integer.MAX_VALUE;
361     for (String nodes : nodes.keySet()) {
362         if (map.containsKey(new Node(objectPoint, nodes)) && weight > map.get(new
Node(objectPoint, nodes))) {
363             weight = map.get(new Node(objectPoint, nodes));
364             returnPoint = nodes;
365         }
366     }
367     return weight;
368 } // 返回这个点周围与他最小权重的点的权重
369
370
371 public void Prim() {
372     if (isEmpty()) return; // 当前集合是空的，就返回
373     if (set.size() != 1) return; // 当前的图存在多个集，返回（不能生成树）
374
375
376     String node = nodes.keySet().toArray(new String[0])[0]; // 获取任意节点
377     flagPoint(node); // 标记这个节点
378
379     // 当不是所有点都被标记，继续执行
380     while (!isAllFlagged()) {
381         HashSet<String> allFlaggedPoint = getAllFlaggedPoint();
382         HashSet<String> allUnFlaggedPoint = getAllUnFlaggedPoint();
383         String newPoint = null; // 新的等待加入的点
384         String selectedPoint = null; // 新的等待加入的点所连接的点
385         int minWeight = Integer.MAX_VALUE;
386         for (String flaggedPoint : allFlaggedPoint) {
387             for (String unFlaggedPoint : allUnFlaggedPoint) {
388                 // 现在遍历nodes
389                 // nodes是点亮的点
390                 // 需要找到距离最近的非点亮点
391                 if (map.containsKey(new Node(flaggedPoint, unFlaggedPoint)) &&
getWeight(flaggedPoint, unFlaggedPoint) < minWeight) {
392                     minWeight = getWeight(flaggedPoint, unFlaggedPoint);

```

```

393         newPoint = unFlaggedPoint;
394         selectedPoint = flaggedPoint;
395     }
396 }
397 }
398     flagPoint(newPoint);
399     System.out.println(selectedPoint + "<->" + newPoint + "=" +
getWeight(selectedPoint, newPoint));
400 }
401 } // Prim 算法
402
403 public int getWeight(String pointA, String pointB) {
404     if (pointA.equals(pointB)) return 0;
405     return map.get(new Node(pointA, pointB));
406 } // 返回两个点的权重
407
408
409 private int getTotalNodeNumber() {
410     return totalNodesNumber;
411 } // 获取点的数量
412
413
414 private int getTotalEdgeNumber() {
415     return flaggedEdgesNumber;
416 } // 获取边的数量
417
418 public void Kruskal() {
419     if (isEmpty()) return; // 当前集合是空的，就返回
420     if (set.size() != 1) return; // 当前的图存在多个集，返回（不能生成树）
421
422     int nodesToBeConnect = nodes.size() - 1;
423     List<Map.Entry<Node, Integer>> weightList = new ArrayList<>(map.entrySet());
424     // 升序，这样我们可以从尾部取出最小的 Entry <点, 边>
425     weightList.sort(Map.Entry.comparingByValue(Comparator.reverseOrder()));
426
427     MyMap tempMap = new MyMap(MyMap.UNDIRECTED_GRAPH);
428     while (nodesToBeConnect > 0) {
429         Node node = weightList.getLast().getKey();
430         // 获取到没有被标记的边
431         // 如果这两个点是连通的，我们就不需要这条边
432         while (tempMap.isConnected(node)) {
433             weightList.removeLast();
434             weightList.removeLast();
435             node = weightList.getLast().getKey();
436         }
437         tempMap.connect(node.A, node.B, 0);
438         nodesToBeConnect--;
439         System.out.println(weightList.getLast().getKey().A + "<->" +
weightList.getLast().getKey().B + "=" + weightList.getLast().getValue());
440         weightList.removeLast(); // 移除这条边
441         weightList.removeLast();
442     }
443 }

```

```

444
445 public void Dijkstra(String a, String b) {
446     if (!nodes.containsKey(a) || !nodes.containsKey(b)) return;
447     int leftNode = getTotalNodeNumber();
448     // 初始包含所有点
449     HashSet<String> unFlaggedPoint = new HashSet<>(nodes.keySet());
450     // chart是图标
451     // 如果一个点没有在chart中，证明它的权重是Infinity
452     // 如果一个点在chart中但是没有在unFlaggedPoint中，那就是这个点被标记
453     HashMap<String, Entry> chart = new HashMap<>();
454
455     String currentPoint = a;
456     // 设置自己到自己的距离为0
457     chart.put(a, new Entry(0, null));
458     unFlaggedPoint.remove(a); // 标记当前点
459     while (leftNode > 0) {
460         // 获取当前点的所有连接点
461         HashMap<String, Integer> allNearbyPoint = getAllNearbyPoint(currentPoint);
462         // 更新每一个nearByPoint
463         for (String s : allNearbyPoint.keySet()) {
464             if (!unFlaggedPoint.contains(s)) continue; // 如果被标记了就跳过这个点
465             // 如果这个点是INF，那么就更新
466             if (!chart.containsKey(s)) {
467                 chart.put(s, new Entry(getWeight(currentPoint, s) +
468 chart.get(currentPoint).weight, currentPoint));
469                 continue;
470             }
471             // 现在这个点不是INF，让我们更新它
472             // 如果 chart(s) > currentPoint 到 s 的值 + chart(current)
473             // 那就更新 chart(s)
474             if (chart.get(s).weight > getWeight(currentPoint, s) +
475 chart.get(currentPoint).weight) {
476                 chart.put(s, new Entry(getWeight(currentPoint, s) +
477 chart.get(currentPoint).weight, currentPoint));
478                 continue;
479             }
480         }
481         unFlaggedPoint.remove(currentPoint);
482         // 现在更新完了，需要从 未标记的 找到最小值
483         String minPoint = null;
484         int minweight = Integer.MAX_VALUE;
485         for (String s : chart.keySet()) {
486             if (!unFlaggedPoint.contains(s)) continue; // 被标记了就继续
487             if (chart.get(s).weight < minweight) {
488                 minweight = chart.get(s).weight;
489                 minPoint = s;
490             }
491         }
492         currentPoint = minPoint;
493         unFlaggedPoint.remove(currentPoint); // 更换点，然后标记为访问
494         leftNode--;
495     }
496     String iterateNode = b;

```

```

494         while (iterateNode != null) {
495             System.out.print("[\" + iterateNode + \"]" + "<---" +
chart.get(iterateNode).weight + "--->");
496             iterateNode = chart.get(iterateNode).prePoint;
497         }
498     }
499
500     public boolean isPointFlagged(String point) {
501         return nodes.get(point);
502     }
503
504     public boolean isEdgeFlagged(String pointA, String pointB) {
505         return isEdgeFlagged(new Node(pointA, pointB));
506     }
507
508     public boolean isEdgeFlagged(Node node) {
509         return edge.get(node);
510     }
511
512     public void BellManFord(String from, String to) {
513         HashMap<String, Integer> distances = new HashMap<>();
514
515         distances.put(from, 0);
516         for (int i = 0; i < getTotalNodeNumber(); i++) {
517             map.forEach((node, weight) -> {
518                 if (distances.containsKey(node.from()) && distances.get(node.from()) +
map.get(node) < distances.getDefault(node.to(), Integer.MAX_VALUE))
519                     distances.put(node.to(), distances.get(node.from()) +
map.get(node));
520             });
521         }
522         // 检查环形边 (重复的)
523         for (int i = 0; i < getTotalNodeNumber(); i++) {
524             map.forEach((node, weight) -> {
525                 if (distances.containsKey(node.from()) && distances.get(node.from()) +
map.get(node) < distances.getDefault(node.to(), Integer.MAX_VALUE))
526                     distances.put(node.to(), -Integer.MAX_VALUE);
527             });
528         }
529
530         System.out.println(distances);
531     }
532
533     public static class Node {
534         private String A;
535         private String B;
536
537         public Node(String a, String b) {
538             A = a;
539             B = b;
540         }
541
542         // getA

```

```

543     public String from() {
544         return A;
545     }
546
547     public void setA(String a) {
548         A = a;
549     }
550
551     // getB
552     public String to() {
553         return B;
554     }
555
556     public void setB(String b) {
557         B = b;
558     }
559
560     @Override
561     public boolean equals(Object o) {
562         if (o == null || getClass() != o.getClass()) return false;
563         Node node = (Node) o;
564         return Objects.equals(A, node.A) && Objects.equals(B, node.B);
565     }
566
567     @Override
568     public int hashCode() {
569         return Objects.hash(A, B);
570     }
571
572     @Override
573     public String toString() {
574         return "Node{" + "A='" + A + '\'' + ", B='" + B + '\'' + '}';
575     }
576 } // 公开内部类
577
578 // String: 当前点
579 // 第二个 Entry : 权重, 以及上一个点
580 // 其中权重指的是 a 到 当前点的最优距离
581 private class Entry {
582     public int weight;
583     public String prePoint;
584
585     public Entry(int weight, String prePoint) {
586
587         this.weight = weight;
588         this.prePoint = prePoint;
589     }
590
591     public int getweight() {
592         return weight;
593     }
594
595     public void setweight(int weight) {

```

```

596         this.weight = weight;
597     }
598
599     public String getPrePoint() {
600         return prePoint;
601     }
602
603     public void setPrePoint(String prePoint) {
604         this.prePoint = prePoint;
605     }
606
607     @Override
608     public boolean equals(Object o) {
609         if (o == null || getClass() != o.getClass()) return false;
610         Entry entry = (Entry) o;
611         return weight == entry.weight && Objects.equals(prePoint, entry.prePoint);
612     }
613
614     @Override
615     public int hashCode() {
616         return Objects.hash(weight, prePoint);
617     }
618 }
619
620 public void Floyd(String from, String to) {
621     HashMap<Node,Integer> distance = new HashMap<>(map); // 最短路径 <Node,weight>
622     // 假设 A --
623     HashMap<Node,String> path = new HashMap<>(); // <A,B> 值是绕行顶点
624     getAllUnFlaggedPoint().forEach(Thru->{
625         getAllUnFlaggedPoint().forEach(From->{
626             if(From.equals(Thru)) return;
627             getAllUnFlaggedPoint().forEach(To->{
628                 if(To.equals(Thru)) return;
629
630                 int from_to = distance.getDefault(new
Node(From,To),Integer.MAX_VALUE);
631
632                 int from_thru = distance.getDefault(new
Node(From,Thru),Integer.MAX_VALUE);
633                 if(from_thru == Integer.MAX_VALUE) return;
634
635                 int thru_to = distance.getDefault(new
Node(Thru,To),Integer.MAX_VALUE);
636                 if(thru_to == Integer.MAX_VALUE) return;
637
638                 if(from_to>from_thru+thru_to)
639                 {
640                     distance.put(new Node(From,To),from_thru+thru_to);
641                     path.put(new Node(From,To),Thru);
642                 }
643             });
644         });
645     });

```

```
646         System.out.println(STR."\{from} -- \{to} == \{distance.getDefault(new
Node(from, to), Integer.MAX_VALUE)}");
647
648         String thru = path.getDefault(new Node(from, to),from);
649         System.out.print(to + "<--");
650         while(thru!=from)
651         {
652             System.out.print(thru + "<--");
653             thru = path.getDefault(new Node(from, thru),from);
654         }
655         System.out.print(from);
656     }
657 }
```

[5] 树

CPT102

二叉搜索树

```
1 public class BinarySearchTree {
2
3     private static ArrayList<Integer> tempList = new ArrayList<>(); // 返回数组
4     private Nodes treeHead; // 一个bst对象有一个树头，初始为null
5
6     BinarySearchTree() { // 空参构造
7     }
8
9
10    // 添加数据
11    public void add(int number) {
12        Nodes search = treeHead; // 初始search是树头，以后就操作search即可
13
14        if (search == null) { // 如果search是空的就创建新节点
15            treeHead = new Nodes();
16            treeHead.data = number;
17        } else { // 非空情况
18            boolean flag = false;
19            while (true) {
20                if (number == search.data) return; // 相等的数据就不执行add操作。
21                if (number < search.data) { // 如果number 小于当前search数据，就往左边找
22                    if (search.left == null) {
23                        flag = false; // 标记flag=false 意思是要添加的数据是search的
24                        break;
25                    } // 如果是尾节点就断开
26                    search = search.left;
27                    continue;
28                }
29                if (number > search.data) { // 如果number 大于当前search数据，往右找
30                    if (search.right == null) { // 如果是尾节点就break
31                        flag = true; // 标记flag=true意思是要添加的数据是search
32                        break;
33                    }
34                    search = search.right;
35                }
36            }
37        }
38
39        Nodes newNode = new Nodes();
40        newNode.middle = search; // 新点的父节点是search
41        newNode.data = number; // 新点数据
42    }
```

```

43
44         if (flag) // flagPoint = true, 意味着添加的数据在search右侧。
45         {
46             search.right = newNode;
47         } else { // flagPoint = false, 意味着添加的数据在search左侧。
48             search.left = newNode;
49         }
50     }
51 }
52
53 // 返回ArrayList;
54 public ArrayList<Integer> getTree() {
55     tempList = new ArrayList<>();
56     getTree(treeHead);
57     return tempList;
58 }
59
60 private void getTree(Nodes nodes) { // 递归的方式中序遍历
61     if (treeHead == null) return;
62     if (nodes.left != null) getTree(nodes.left);
63     tempList.add(nodes.data);
64     if (nodes.right != null) getTree(nodes.right);
65 }
66
67 // 查询是否包含某个点
68 public boolean contains(int number) {
69     return contains(treeHead, number);
70 }
71
72 private boolean contains(Nodes node, int number) {
73     if (node == null) return false;
74     if (number == node.data) return true;
75     if (number < node.data) return contains(node.left, number);
76     // if(number > node.data)
77     return contains(node.right, number);
78 }
79
80 // 删除节点操作
81 public boolean delete(int number) {
82
83     if (treeHead == null) return false; // 如果当前对象没有树，那么直接返回false，删除失
败。
84     Nodes search = treeHead; // 让search从头节点开始操作；search代表要删除的
点
85     boolean flag = false; // search是上一个点的左边还是右边：false ->
left; true->right
86     while (search != null) { // 找到要删除的点
87         if (number == search.data) break; // 找到就break
88         if (number > search.data) { // 如果number大于当前search，那么往右边找。
89             flag = true; // flagPoint = true代表往右找
90             search = search.right;
91             continue; // 一定要continue否则会下面if冲突
92         }

```

```

93         if (number < search.data) {
94             flag = false;           // 往左边找，标记flag
95             search = search.left;
96         }
97     }
98     // 如果不存在就返回false
99     if (search == null) return false;
100
101     // 如果要删除的是头节点，而且该对象仅有头节点，那么头节点为null即可
102     if (search == treeHead && treeHead.left == null && treeHead.right == null) {
103         treeHead = null;
104         return true;
105     }
106
107
108
109     // 情况1： 叶子节点
110     if (search.left == null && search.right == null) {
111         if (flag) { // 删除右边
112             search.middle.right = null;
113         } else { // 删除左边
114             search.middle.left = null;
115         }
116
117         search.middle = null;
118         return true;
119     }
120
121
122     // 情况2： 只有右子树
123     if (search.left == null && search.right != null) {
124         //如果是头节点， 直接让头节点指向下一个即可
125         if (search == treeHead) {
126             treeHead = treeHead.right;
127             treeHead.middle = null; //取消引用，直接回收内存
128         } else if (flag) { // 右边
129             search.middle.right = search.right;
130             search.right.middle = search.middle;
131
132         } else { // 左边
133             search.middle.left = search.right;
134             search.right.middle = search.middle;
135         }
136
137         search.middle = null;
138         search.left = null;
139         search.right = null;
140         return true;
141     }
142     // 情况3： 只有左子树
143     if (search.left != null && search.right == null) {
144         if (search == treeHead) {
145             treeHead = treeHead.left;

```

```

146         treeHead.middle = null;
147     } else if (flag) { // 右边
148         search.middle.right = search.left;
149         search.left.middle = search.middle;
150     } else { // 左边
151         search.middle.left = search.left;
152         search.left.middle = search.middle;
153     }
154
155     search.middle = null;
156     search.left = null;
157     search.right = null;
158     return true;
159 }
160
161 // 情况4: 既有左子树也有右子树。
162 if (search.left != null && search.right != null) {
163     boolean sign = false; // sign用与判断search节点在上一个节点的左还是右。默认
false= 左, 如果true则为右
164     Nodes temp = search; // temp是search左子树中最大的点
165     temp = temp.left;
166     while (temp.right != null) { // 让temp作为search左子树中最大的节点
167         temp = temp.right;
168         sign = true;
169     }
170
171     // 如果 temp 有左节点 , 就晋升左节点到 temp 位置
172     Nodes tempFather = temp.middle; // 记录temp的父节点
173     Nodes leftTree = temp.left; // 记录temp左子树
174
175     if (leftTree != null) leftTree.middle = null; // 左子树和temp断开
176     temp.left = null; // temp左右子树都是null (temp不可能有右子树)
177
178     // 断开temp和父节点
179     if (sign) {
180         temp.middle.right = null;
181     } else {
182         temp.middle.left = null;
183     }
184     temp.middle = null;
185
186     // 让左子树晋升
187     if (leftTree != null) {
188         leftTree.middle = tempFather;
189         if (sign) {
190             tempFather.right = leftTree;
191         } else {
192             tempFather.left = leftTree;
193         }
194     }
195
196     // 现在要让temp和search换位
197     temp.middle = search.middle;

```

```

198         temp.left = search.left;
199         temp.right = search.right;
200
201         // 如果search这个点不是treeHead，就更改其middle
202         if (search.middle != null) {
203             if (flag) { // flagPoint 代表search是上一个点的左边还是右边，true代表search
在  上一个点的右边
204                 search.middle.right = temp;
205             } else { // search在上一个点的左边
206                 search.middle.left = temp;
207             }
208         }
209
210         // 让search的左右子树指向temp
211         if (search.left != null) search.left.middle = temp;
212         if (search.right != null) search.right.middle = temp;
213
214         // 如果更改的是树头，就让树头 = temp;
215         if (search == treeHead) treeHead = temp;
216
217
218         return true;
219     }
220
221     return false;
222 }
223
224 private static class Nodes {
225     Nodes left;
226     Nodes middle;
227     Nodes right;
228     int data;
229 }
230 }

```

AVL 树

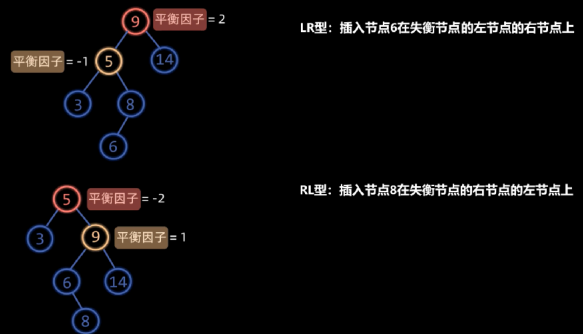
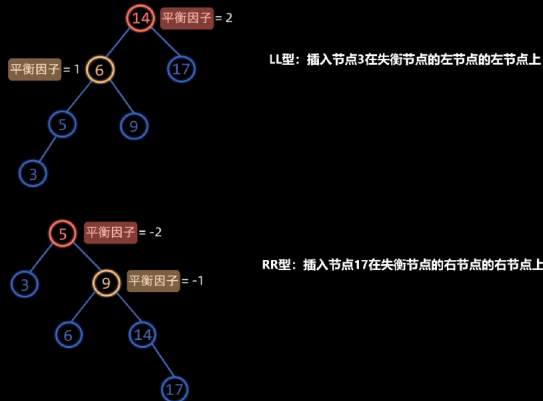
CPT102

可以观看[这个视频了解AVL树。](#)

四种失衡情况

LL型	RR型	LR型	RL型
失衡结点: 平衡因子 = 2	失衡结点: 平衡因子 = -2	失衡结点: 平衡因子 = 2	失衡结点: 平衡因子 = -2
失衡结点左孩子: 平衡因子 = 1	失衡结点右孩子: 平衡因子 = -1	失衡结点左孩子: 平衡因子 = -1	失衡结点右孩子: 平衡因子 = 1
右旋	左旋	左旋左孩子, 然后右旋	右旋右孩子, 然后左旋

*这里的旋转操作指的是对失衡节点执行旋转



```

1  import java.util.ArrayList;
2  import java.util.Stack;
3
4  /*
5  * 1. 点A的平衡因子定义如下: A的左子树高度 - A的右子树高度
6  * 如果子树为null, 那么计算而言, 高度视为0
7  * 2. 失衡节点因子=2, 其左节点因子=1; 对失衡节点进行左旋      LL
8  * 失衡节点因子=2, 左节点因子=-1; 左节点左旋后, 节点右旋      LR
9  * 失衡节点因子=-2, 其右节点因子=-1; 对失衡节点进行左旋      RR
10 * 失衡节点因子=-2, 其右节点因子=1; 右节点右旋, 节点左旋      RL
11 */
12 public class AVLTree {
13     private Nodes treeHead; // 树头
14
15
16
17
18     public AVLTree() {
19     } // 无参构造
20     private static class Nodes {
21         int key; // 当前节点值
22         Nodes left; // 左子树
23         Nodes right; // 右子树
24         Nodes middle; // 上一级
25         int height = 1; // 把当前节点作为一棵树, 这棵树的高度
26     } // 节点
27
28
29     // 插入
30     public boolean insert(int number) {
31         // 树头不存在
32         if (treeHead == null)
33             {

```

```

34         treeHead = new Nodes();
35         treeHead.key = number;
36         return true;
37     }
38     // 树头存在
39     // search是作为临时搜寻的。其作用是充当待插入节点的父节点
40     Nodes search = treeHead;
41     // 寻找插入节点的父节点
42     while(search!=null)
43     {
44         // 如果这个值存在，就不插入数据，返回false
45         if(number == search.key) return false;
46
47         // 如果数字大于当前节点，就需要往右边找
48         // 如果需要往右找，但是右边是null，直接break
49         if(number>search.key)
50         {
51             if(search.right==null) break;
52             search = search.right;
53             continue;
54         }
55         // 如果数字小于当前节点，就需要往左边找
56         // 同理如果左边是null，直接break
57         if(number<search.key)
58         {
59             if(search.left==null) break;
60             search = search.left;
61         }
62     }
63
64     // 现在我们的search就待插入节点的父节点。
65
66     // 什么时候需要更新父节点的高度？
67     //     当数据插入父节点的子节点，例如插入父节点的左边，
68     //     但是父节点的另一边（右边）是空的，那么父节点就要更新。
69     //     父节点的父节点就要判断谁大，一路比下去
70
71     // 应当插入左节点的情况
72     // 先申请一个节点，赋值，作为左节点
73     // 左节点指向search这个父节点
74     // 如果这个树的高度变了（另一侧为null），就更新父节点search的高度，并且迭代上去
75     if(number<search.key)
76     {
77         search.left = new Nodes();
78         search.left.key = number;
79         search.left.middle = search;
80         if(search.right==null) updateHeight(search);
81         executeRotateOperation(search.left);
82         return true;
83     }
84
85     // 这里就是应当插入右节点的情况
86     // 申请一个节点，赋值，作为右节点

```

```

87 // 右节点指向search这个父节点
88 // 同理如果树增长了就更新
89 if(number>search.key)
90 {
91     search.right = new Nodes();
92     search.right.key = number;
93     search.right.middle = search;
94     if(search.left==null) updateHeight(search);
95     executeRotateOperation(search.right);
96     return true;
97 }
98 return false;
99 }
100 private void executeRotateOperation(Nodes node){
101     if(node == null) return;
102     // 一直寻找到失衡节点(往插入节点的父节点找)
103     while(node!=null&&Math.abs(getBalanceFactor(node))<=1)
104     {
105         node = node.middle;
106     }
107     if(node == null) return;
108     /*if(Math.abs(getBalanceFactor(node))>2)
109     {
110         System.out.println("Error: diff abs is larger than 2. Diff = " +
111 Math.abs(getBalanceFactor(node.left) - getBalanceFactor(node.right)));
112         return;
113     }*/
114     // LL
115     if(getBalanceFactor(node)==2&&getBalanceFactor(node.left)==1)
116     {
117         rightRotate(node);
118         return;
119     }
120     // RR
121     if(getBalanceFactor(node)==-2&&getBalanceFactor(node.right)==-1)
122     {
123         leftRotate(node);
124         return;
125     }
126     // LR
127     if(getBalanceFactor(node)==2&&getBalanceFactor(node.left)==-1)
128     {
129         leftRotate(node.left);
130         rightRotate(node);
131         return;
132     }
133     // RL
134     if(getBalanceFactor(node)==-2&&getBalanceFactor(node.right)==1)
135     {
136         rightRotate(node.right);
137         leftRotate(node);
138         return;
139     }

```



```

139         // System.out.println("Error: didn't find suitable rotation operation.");
140     }
141     private static void updateHeight(Nodes search){
142         while(search!=null){
143             int leftHeight = 0;
144             if(search.left!=null) leftHeight = search.left.height+1;
145             int rightHeight = 0;
146             if(search.right!=null) rightHeight = search.right.height+1;
147             search.height = max(max(1,leftHeight),rightHeight);
148             search = search.middle;
149         }
150     }
151
152
153     private static int max(int a, int b){
154         return Math.max(a,b);
155     }
156
157     private static ArrayList<Integer> list = new ArrayList<>();
158     private static Stack<Nodes> stack = new Stack<>();
159     public ArrayList<Integer> getTree(){
160         list = new ArrayList<>();
161         getTree(treeHead);
162         return list;
163     }
164     private void getTree(Nodes node){
165         if(node==null) return;
166         getTree(node.left);
167         list.add(node.key);
168         getTree(node.right);
169     }
170     private static int getBalanceFactor(Nodes node){
171         if(node == null) return 0;
172         int leftHeight = 0;
173         if(node.left!=null) leftHeight = node.left.height;
174         int rightHeight = 0;
175         if(node.right!=null) rightHeight = node.right.height;
176         return leftHeight-rightHeight;
177     }
178     private void leftRotate(Nodes node){
179         // 定义父节点和冲突左节点
180         Nodes fatherNode = node.middle;
181         Nodes conflictLeftNode = node.right.left;
182         // 断开父节点和支点的联系，首先判断大小来断定左右
183         // false 代表我们选转的支点在父节点的左边
184         // true 代表我们旋转的支点在父节点的右边
185         boolean flag = (fatherNode != null && node.key > fatherNode.key);
186         if(fatherNode!=null)
187         {
188             if(flag) fatherNode.right = null;
189             else fatherNode.left = null;
190         }
191         node.middle = null;

```

```

192
193 // 首先检查支点的右节点有无左节点。要处理冲突问题
194 // 如果说没有左节点，简单左旋
195 if(conflictLeftNode==null)
196 {
197     // 断开 node 和 node.right 的联系
198     Nodes rightNode = node.right;
199     rightNode.middle = null;
200     node.right = null;
201
202     // 让fatherNode和 rightNode连接起来
203     if(fatherNode != null)
204     {
205         rightNode.middle = fatherNode;
206         if(flag)fatherNode.right = rightNode;
207         else fatherNode.left = rightNode;
208     }
209
210     // 把node和 node.right连接起来
211     node.middle = rightNode;
212     rightNode.left = node;
213
214     // 如果fatherNode == null 意味着其实node就是fatherNode，那么更改树头
215     if(fatherNode == null)
216         treeHead = rightNode;
217
218     // 最后，调整高度：
219     updateHeight(node);
220
221     return;
222 }
223 // 如果有左节点，复杂左旋
224 if(conflictLeftNode!=null)
225 {
226     // 断开 node 和 node.right 的联系
227     Nodes rightNode = node.right;
228     rightNode.middle = null;
229     node.right = null;
230
231     // 让fatherNode和 rightNode连接起来
232     if(fatherNode != null)
233     {
234         rightNode.middle = fatherNode;
235         if(flag)fatherNode.right = rightNode;
236         else fatherNode.left = rightNode;
237     }
238     // 切除冲突左子树
239     conflictLeftNode.middle = null;
240     rightNode.left = null;
241     // 冲突树连接到node右边
242     node.right = conflictLeftNode;
243     conflictLeftNode.middle = node;
244     // 把node和 node.right连接起来

```

```

245         node.middle = rightNode;
246         rightNode.left = node;
247
248         // 如果fatherNode == null 意味着其实node就是fatherNode，那么更改树头
249         if(fatherNode == null)
250             treeHead = rightNode;
251
252         // 最后，调整高度：
253         updateHeight(conflictLeftNode);
254
255
256
257         return;
258     }
259 }
260 private void rightRotate(Nodes node){
261     // 定义父节点和冲突右节点
262     Nodes fatherNode = node.middle;
263     Nodes conflictRightNode = node.left.right;
264
265     // 先判断支点在父节点的左边还是右边
266     // false 是左边， true是右边
267     boolean flag = (fatherNode!=null && node.key>fatherNode.key);
268
269     // 断开父节点和支点的联系
270     if(fatherNode!=null)
271     {
272         if(flag) fatherNode.right=null;
273         else fatherNode.left = null;
274     }
275     node.middle = null;
276
277     // 如果无冲突右节点，简单右旋
278     if(conflictRightNode==null)
279     {
280         // 断开 node 和 node.left的联系
281         Nodes leftNode = node.left;
282         leftNode.middle = null;
283         node.left = null;
284
285         // 让 fatherNode 和 leftNode连起来
286         if(fatherNode!=null)
287         {
288             leftNode.middle = fatherNode;
289             if(flag) fatherNode.right = leftNode;
290             else fatherNode.left = leftNode;
291         }
292
293         // 把node和leftNode连起来
294         node.middle = leftNode;
295         leftNode.right = node;
296
297         // 如果fatherNode == null 意味着其实node就是fatherNode，那么更改树头

```

```

298         if(fatherNode == null)
299             treeHead = leftNode;
300
301         // 最后，调整高度：
302         updateHeight(node);
303
304         return;
305     }
306     // 如果有用冲突右节点
307     if(conflictRightNode!=null)
308     {
309         // 断开 node 和 node.left的联系
310         Nodes leftNode = node.left;
311         leftNode.middle = null;
312         node.left = null;
313
314         // 让 fatherNode 和 leftNode连起来
315         if(fatherNode!=null)
316         {
317             leftNode.middle = fatherNode;
318             if(flag) fatherNode.right = leftNode;
319             else fatherNode.left = leftNode;
320         }
321         // 切除冲突右节点
322         conflictRightNode.middle = null;
323         leftNode.right = null;
324         // 冲突右节点连接到node左节点上
325         node.left = conflictRightNode;
326         conflictRightNode.middle=node;
327         // 把node和leftNode连起来
328         node.middle = leftNode;
329         leftNode.right = node;
330
331         // 如果fatherNode == null 意味着其实node就是fatherNode，那么更改树头
332         if(fatherNode == null)
333             treeHead = leftNode;
334
335         // 最后，调整高度：
336         updateHeight(conflictRightNode);
337
338         return;
339     }
340 }
341
342 public boolean remove(int number) {
343
344     if (treeHead == null) return false; // 如果当前对象没有树，那么直接返回false，删除失败。
345
346     Nodes node = treeHead; // 让search从头节点开始操作；search代表要删除的点
347     boolean flag = false; // search是上一个点的左边还是右边：false ->
348     left; true->right
349     while (node != null) { // 找到要删除的点
350         if (number == node.key) break; // 找到就break

```

```

349         if (number > node.key) {           // 如果number大于当前search，那么往右边找。
350             flag = true;                     // flagPoint = true代表往右找
351             node = node.right;
352             continue;                       // 一定要continue否则会下面if冲突
353         }
354         if (number < node.key) {
355             flag = false;                    // 往左边找，标记flag
356             node = node.left;
357         }
358     }
359     // 如果不存在就返回false
360     if (node == null) return false;
361
362     // 如果要删除的是头节点，而且该对象仅有头节点，那么头节点为null即可
363     if (node == treeHead && treeHead.left == null && treeHead.right == null) {
364         treeHead = null;
365         return true;
366     }
367
368
369
370     // 情况1： 叶子节点
371     if (node.left == null && node.right == null) {
372         if (flag) { // 删除右边
373             node.middle.right = null;
374         } else { // 删除左边
375             node.middle.left = null;
376         }
377
378         // 取消依赖
379         node.middle = null;
380
381         // 取下父节点
382         Nodes fatherNode = node.middle;
383         // 更新父节点高度
384         updateHeight(fatherNode);
385         // 执行旋转(如果有)
386         executeRotateOperation(fatherNode);
387
388         return true;
389     }
390
391
392     // 情况2： 只有右子树
393     if (node.left == null && node.right != null) {
394         //如果是头节点， 直接让头节点指向下一个即可
395         if (node == treeHead) {
396             treeHead = treeHead.right;
397             treeHead.middle = null; //取消引用，直接回收内存
398         } else if (flag) { // 右边
399             node.middle.right = node.right;
400             node.right.middle = node.middle;
401

```

```

402         } else { // 左边
403             node.middle.left = node.right;
404             node.right.middle = node.middle;
405         }
406
407         // 取下父节点
408         Nodes fatherNode = node.middle;
409         // 更新父节点高度
410         updateHeight(fatherNode);
411         // 执行旋转(如果有)
412         executeRotateOperation(fatherNode);
413
414         node.middle = null;
415         node.left = null;
416         node.right = null;
417         return true;
418     }
419     // 情况3: 只有左子树
420     if (node.left != null && node.right == null) {
421         if (node == treeHead) {
422             treeHead = treeHead.left;
423             treeHead.middle = null;
424         } else if (flag) { // 右边
425             node.middle.right = node.left;
426             node.left.middle = node.middle;
427         } else { // 左边
428             node.middle.left = node.left;
429             node.left.middle = node.middle;
430         }
431
432         // 取下父节点
433         Nodes fatherNode = node.middle;
434         // 更新父节点高度
435         updateHeight(fatherNode);
436         // 执行旋转(如果有)
437         executeRotateOperation(fatherNode);
438
439         node.middle = null;
440         node.left = null;
441         node.right = null;
442         return true;
443     }
444
445     // 情况4: 既有左子树也有右子树。
446     if (node.left != null && node.right != null) {
447         boolean sign = false; // sign用与判断search节点在上一个节点的左还是右。默认
448         // false= 左, 如果true则为右
449         Nodes temp = node; // temp是search左子树中最大的点
450         temp = temp.left;
451         while (temp.right != null) { // 让temp作为search左子树中最大的节点
452             temp = temp.right;
453             sign = true;
454         }

```

```

454
455 // 如果 temp 有左节点，就晋升左节点到 temp 位置
456 Nodes tempFather = temp.middle; // 记录temp的父节点
457 Nodes leftTree = temp.left; // 记录temp左子树
458
459 if (leftTree != null) leftTree.middle = null; // 左子树和temp断开
460 temp.left = null; // temp左右子树都是null (temp不可能有右子树)
461
462 // 断开temp和父节点
463 if (sign) {
464     temp.middle.right = null;
465 } else {
466     temp.middle.left = null;
467 }
468 temp.middle = null;
469
470 // 让左子树晋升
471 if (leftTree != null) {
472     leftTree.middle = tempFather;
473     if (sign) {
474         tempFather.right = leftTree;
475     } else {
476         tempFather.left = leftTree;
477     }
478 }
479
480 // 现在要让temp和search换位
481 temp.middle = node.middle;
482 temp.left = node.left;
483 temp.right = node.right;
484
485 // 如果search这个点不是treeHead，就更改其middle
486 if (node.middle != null) {
487     if (flag) { // flagPoint 代表search是上一个点的左边还是右边，true代表search
在上一个点的右边
488         node.middle.right = temp;
489     } else { // search在上一个点的左边
490         node.middle.left = temp;
491     }
492 }
493
494 // 让search的左右子树指向temp
495 if (node.left != null) node.left.middle = temp;
496 if (node.right != null) node.right.middle = temp;
497
498 // 如果更改的是树头，就让树头 = temp;
499 if (node == treeHead) treeHead = temp;
500
501 // 取下父节点
502 Nodes fatherNode = node.middle;
503 // 更新父节点高度
504 updateHeight(fatherNode);
505 // 执行旋转(如果有)

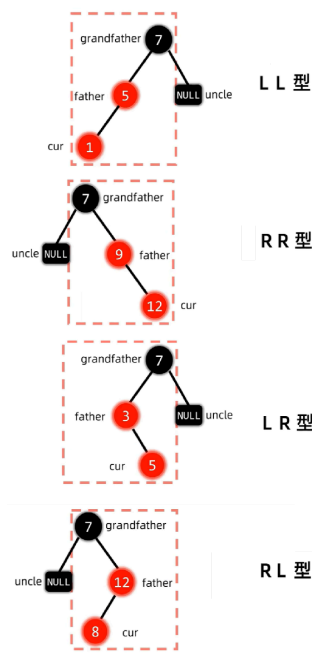
```

```
506         executeRotateOperation(fatherNode);
507
508         return true;
509     }
510
511     return false;
512 }
513
514 public boolean contains(int number){
515     Nodes node = treeHead;
516     while(node!=null)
517     {
518         if(node.key == number) return true;
519         if(number>node.key)
520         {
521             node = node.right;
522             continue;
523         }
524         if(number<node.key)
525         {
526             node = node.left;
527             continue;
528         }
529     }
530     return false;
531 }
532 }
```

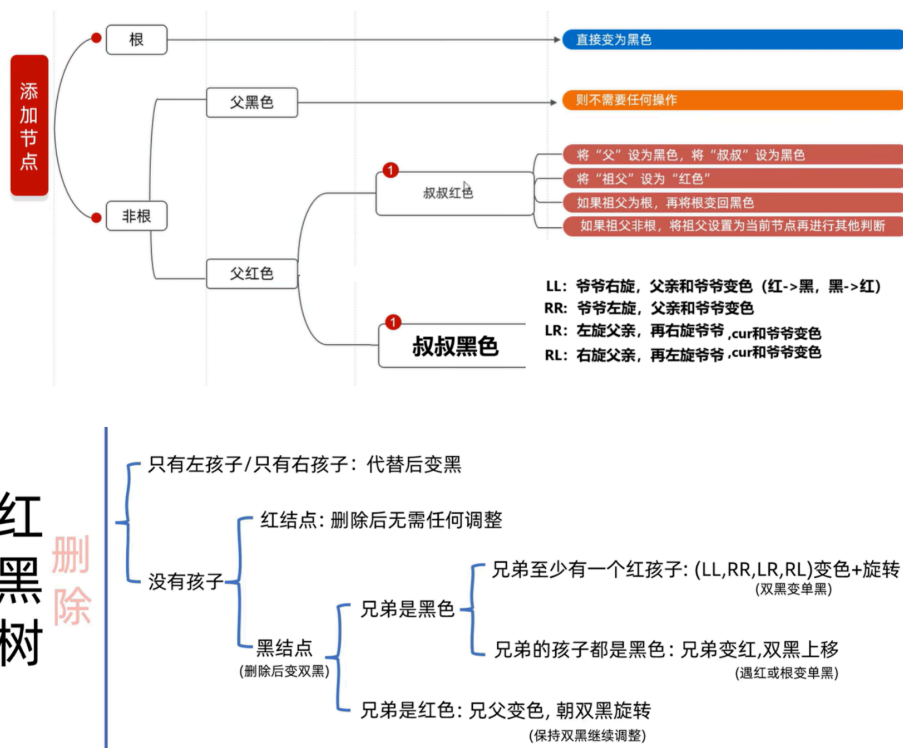
红黑树

额外内容

观看[这个视频](#)



红黑树删除



```

1  import java.util.ArrayList;
2
3  public class RedBlackTree {
4      private static final int LL = 0;
5      private static final int LR = 1;
6      private static final int RL = 2;
7      private static final int RR = 3;
8      private static int componentNumber = 0;
9      private static ArrayList<Integer> list;
10     private Nodes treeHead;
11
12     public RedBlackTree() {
13     } // 空参构造
14
15
16     // [Public] 查看是否是空树
17     public boolean isEmpty() {
18         return treeHead == null;
19     }
20
21     // [Public] 查看是否存在这个值
22     public boolean contains(int key) {
23         return findThisNode(key) != null;
24     }
25
26     // [Public] 查找并返回这个点 (如果没有返回就是null)
27     private Nodes findThisNode(int key) {
28         Nodes search = treeHead;
29         while (search != null) {
30             if (key == search.key) return search;
31             if (key > search.key) {

```

```

32         search = search.right;
33         continue;
34     }
35     if (key < search.key) {
36         search = search.left;
37         continue;
38     }
39 }
40 return search;
41 }
42
43 // [Public] 插入
44 public void insert(int key) {
45     if (treeHead == null) {
46         insertTreeHead(key); // 给树头插入数据
47         componentNumber++;
48         return;
49     }
50
51     Nodes search = treeHead;
52     Nodes fatherNode = null; // 这个fatherNode 是search的父亲节点
53
54     while (search != null) {
55         if (key == search.key) return; // 存在了，不插入
56         if (key > search.key) {
57             fatherNode = search;
58             search = search.right;
59             continue;
60         }
61         if (key < search.key) {
62             fatherNode = search;
63             search = search.left;
64             continue;
65         }
66     }
67     // 执行完之后，search就是我们要插入数据的地方
68     search = new Nodes(); // 申请新节点
69     search.key = key; // 更新数值
70     connectNodes(fatherNode, search);
71     componentNumber++;
72
73     rule(search); // 按照红黑树规则更新节点
74 }
75
76 // [Public] 返回这棵树
77 public ArrayList<Integer> getTree() {
78     list = new ArrayList<>();
79     getTree(treeHead);
80     return list;
81 }
82 // [Public] 获取这棵树元素个数
83
84 public int getComponentNumber() {

```

```

85         return componentNumber;
86     }
87
88     // [Public] 删除
89     public void delete(int key) {
90         Nodes node = findThisNode(key);
91         if (node == null) return; // 如果这个节点不存在，就不用删除
92         delete(node);
93         componentNumber--;
94     }
95
96     // 函数重载用于内部
97     // 传入节点不可能为null
98     private void delete(Nodes node) {
99         // 特殊情况删除树头
100        if (leaf(node) && node == treeHead) {
101            treeHead = null;
102            return;
103        }
104
105        // 如果这个节点是叶子节点（复杂情况）
106        if (leaf(node)) {
107            deleteOperationA(node);
108            return;
109        }
110        // 如果这个节点只有singleBranch，执行操作B
111        if (singleBranch(node)) {
112            deleteOperationB(node);
113            return;
114        }
115        // 如果运行到这里就证明这个节点左右都有节点
116        deleteOperationH(node);
117    }
118
119
120    private void deleteOperationH(Nodes node) {
121        Nodes min = findMinNode(node.right); // 找左边最大的 or 右边最小的
122        delete(min);
123        copyKeyProperties(node, min);
124    }
125
126    // 把 source中的所有属性覆盖到target上（除了节点left,right,middle和color，这个是属于树的属性）
127    // 如果以后Node 更新了更多属性例如<E>那么这里就需要更改
128    private void copyKeyProperties(Nodes target, Nodes source) {
129        target.key = source.key;
130    }
131
132    // 寻找当前节点下最大的节点（包括自身）
133    private Nodes findMaxNode(Nodes node) {
134        Nodes finalReturn = node;
135        while (finalReturn.right != null) finalReturn = finalReturn.right;
136        return finalReturn;

```

```

137     }
138
139     // 寻找当前节点下最小的节点（包括自身）
140     private Nodes findMinNode(Nodes node) {
141         Nodes finalReturn = node;
142         while (finalReturn.left != null) finalReturn = finalReturn.left;
143         return finalReturn;
144     }
145
146     // 这个节点只有左子树或者右子树。
147     // 直接晋升孩子+转孩子为黑色
148     private void deleteOperationB(Nodes node) {
149         // 只有左子树
150         if (node.left != null) {
151             Nodes middleNode = node.middle;
152             Nodes leftNode = node.left;
153
154             disconnectNodes(middleNode, node);
155             disconnectNodes(node, leftNode);
156
157             if (node == treeHead) treeHead = leftNode;
158             else connectNodes(middleNode, leftNode);
159
160             leftNode.color = false; // 晋升节点需要转为黑色
161         } else { // 只有右子树
162             Nodes middleNode = node.middle;
163             Nodes rightNode = node.right;
164
165             disconnectNodes(middleNode, node);
166             disconnectNodes(node, rightNode);
167
168             if (node == treeHead) treeHead = rightNode;
169             else connectNodes(middleNode, rightNode);
170             rightNode.color = false; // 晋升节点需要转为黑色
171         }
172         // componentNumber--;
173     }
174
175     // 如果这个节点是singleBranch的，则返回True否则返回false
176     private boolean singleBranch(Nodes node) {
177         return (node.left == null && node.right != null) || (node.left != null &&
node.right == null);
178     }
179
180     // 如果这个节点是叶子节点（没有孩子）
181     private void deleteOperationA(Nodes node) {
182         if (getColor(node)) // 如果是红节点，直接删除
183         {
184             Nodes fatherNode = node.middle;
185             disconnectNodes(fatherNode, node);
186             // componentNumber -- ;
187             return;
188         }

```

```

189 // 接下来，是删除黑色叶子节点的操作
190 deleteOperationD(node, true); // 从这里传入的node不可能是null
191 }
192
193 // 这个操作仅仅用于删除黑色节点(或调整)，而且这个黑色节点是叶子节点（无左右节点）
194 private void deleteOperationD(Nodes node, boolean delete) {
195     if (node == treeHead && !delete) // 如果传递到树头，而且是调整模式（不删除），直接转为
黑色return
196     {
197         node.color = false;
198         return;
199     }
200     Nodes brotherNode = getBrother(node); // 这个brotherNode可能是null
201     if (brotherNode == null) {
202         return;
203     }
204     if (getColor(brotherNode)) {
205         // 兄弟节点是红色的
206         deleteOperationG(node, brotherNode, delete);
207         return;
208     }
209     // 这个兄弟节点是黑色的
210
211     if (atLeastOneRedChild(brotherNode))
212         // 如果兄弟至少有一个红色孩子，就执行deleteOperationE
213         deleteOperationE(node, brotherNode, delete);
214     else
215         // 反之如果兄弟都是黑色孩子，就执行deleteOperationF
216         deleteOperationF(node, brotherNode, delete);
217 }
218
219 private void deleteOperationG(Nodes node, Nodes brotherNode, boolean delete) {
220     // 兄弟父亲变色
221     switchColor(brotherNode);
222     switchColor(node.middle);
223
224     Nodes fatherNode = node.middle;
225     boolean rotate = getPosition(fatherNode, node);
226     // 父节点朝着node方向旋转
227     if (rotate) rightRotateNode(fatherNode);
228     else leftRotateNode(fatherNode);
229
230     deleteOperationD(node, false); // 要删除的节点是node，但现在先保留，继续调整
231     // 调整完成后，就删除
232     if (delete) {
233         // componentNumber--;
234         disconnectNodes(fatherNode, node);
235     }
236 }
237
238 // 传入brother节点
239 private void deleteOperationE(Nodes node, Nodes brotherNode, boolean delete) {
240     switch (getPattern(brotherNode)) {

```

```

241     case LL -> {
242         // 颜色更新
243         brotherNode.left.color = getColor(brotherNode);
244         brotherNode.color = getColor(brotherNode.middle);
245         brotherNode.middle.color = false;
246         // 右旋转父节点
247         rightRotateNode(node.middle);
248         // 删除目标节点
249         if (delete) {
250             // componentNumber--;
251             disconnectNodes(node.middle, node);
252         }
253     }
254     case RR -> {
255         // 颜色更新
256         brotherNode.right.color = getColor(brotherNode);
257         brotherNode.color = getColor(brotherNode.middle);
258         brotherNode.middle.color = false;
259         // 左旋转父节点
260         leftRotateNode(node.middle);
261         // 删除目标节点
262         if (delete) {
263             // componentNumber--;
264             disconnectNodes(node.middle, node);
265         }
266     }
267     case LR -> {
268         // 颜色更新
269         brotherNode.right.color = getColor(brotherNode.middle);
270         brotherNode.middle.color = false;
271         // 左旋 brother
272         leftRotateNode(brotherNode);
273         // 右旋父节点
274         rightRotateNode(node.middle);
275         // 删除目标节点
276         if (delete) {
277             // componentNumber--;
278             disconnectNodes(node.middle, node);
279         }
280     }
281     case RL -> {
282         brotherNode.left.color = getColor(brotherNode.middle);
283         brotherNode.middle.color = false;
284         rightRotateNode(brotherNode);
285         leftRotateNode(node.middle);
286         if (delete) {
287             // componentNumber--;
288             disconnectNodes(node.middle, node);
289         }
290     }
291 }
292 }
293

```

```

294     private void deleteOperationF(Nodes node, Nodes brotherNode, boolean delete) {
295         brotherNode.color = true; // 兄弟变红
296         Nodes fatherNode = node.middle;
297         boolean fatherNodeColor = getColor(fatherNode);
298         if (delete) {
299             // componentNumber--;
300             disconnectNodes(fatherNode, node);
301         }
302         if (fatherNodeColor) // 双黑节点上移,
303             fatherNode.color = false; // 如果双黑移动到红色节点, 直接变黑。结束
304         else deleteOperationD(fatherNode, false); // 不删除只调整
305     }
306
307     // 返回四种形态之一, 传入的node必须为中间节点
308     private int getPattern(Nodes node) {
309         if (getPosition(node.middle, node) && getColor(node.right)) return RR;
310
311         if (!getPosition(node.middle, node) && getColor(node.left)) return LL;
312
313         if (getPosition(node.middle, node) && !getColor(node.right) &&
141         getColor(node.left)) return RL;
314
315         if (!getPosition(node.middle, node) && getColor(node.right) &&
!getColor(node.left)) return LR;
316
317         System.err.println("NO PATTERN FOUND!");
318         return -1;
319     }
320
321     // 如果这个节点至少有一个孩子是红色, 就返回true
322     // 也就是如果俩孩子都是黑色就返回false
323     private boolean atLeastOneRedChild(Nodes node) {
324         return (node.left != null && node.left.color) || (node.right != null &&
node.right.color);
325     }
326
327     // 返回这个节点的兄弟节点
328     private Nodes getBrother(Nodes node) {
329         // 特殊
330         if (node.middle != null && node.key == node.middle.key) {
331             if (node.middle.right == node) return node.middle.left;
332             else return node.middle.right;
333         }
334         // 一般
335         if (getPosition(node.middle, node)) return node.middle.left;
336         else return node.middle.right;
337     }
338
339     // 如果这个节点是单节点就返回 True, 否则返回false
340     private boolean leaf(Nodes node) {
341         if (node == null) return false;
342         return node.left == null && node.right == null;
343     }

```

```

344
345 private void getTree(Nodes nodes) {
346     if (nodes == null) return;
347     getTree(nodes.left);
348     list.add(nodes.key);
349     getTree(nodes.right);
350 }
351
352 // 按照红黑树规则更新 node 节点
353 private void rule(Nodes node) {
354     if (node == treeHead) return; // 如果是根节点，不执行任何操作
355
356     // 接下来是非根节点
357     if (!getColor(node.middle)) return; // 如果父节点是黑色，不执行任何操作
358
359     // 接下来是非根节点且父节点是红色
360
361     Nodes uncleNode = getUncleNodes(node); // 叔叔节点
362     if (getColor(uncleNode)) // 如果叔叔节点是红色，执行optionA。需要考虑递归
363     {
364         insertOperationA(node);
365         return;
366     }
367     // 接下来是非根节点且父节点是红色，叔叔节点是黑色
368     Nodes fatherNode = node.middle;
369     Nodes grandfatherNode = node.middle.middle;
370     switch (getPattern(node.middle)) {
371         case LL -> {
372             rightRotateNode(grandfatherNode); // 爷爷右旋
373             switchColor(fatherNode); // 转变颜色 适用于LL和RR
374             switchColor(grandfatherNode);
375         }
376         case RR -> {
377             leftRotateNode(grandfatherNode);
378
379             switchColor(fatherNode);
380             switchColor(grandfatherNode);
381         }
382         case LR -> {
383             leftRotateNode(fatherNode); // 左旋父亲
384             rightRotateNode(grandfatherNode); // 右旋爷爷
385
386             switchColor(node); // 转变颜色 适用于LR和RL
387             switchColor(grandfatherNode);
388         }
389         case RL -> {
390             rightRotateNode(fatherNode);
391             leftRotateNode(grandfatherNode);
392
393             switchColor(node);
394             switchColor(grandfatherNode);
395         }
396     }

```



```

397
398     }
399
400     private void switchColor(Nodes node) {
401         if (node != null) node.color = !node.color;
402     }
403
404     // 左旋转node
405     private void leftRotateNode(Nodes node) {
406         Nodes fatherNode = node.middle;
407         Nodes leftNode = node.left;
408         Nodes rightNode = node.right;
409
410
411         Nodes subNode = null; // 这个是冲突的节点（如果不为null）
412         if (rightNode != null) subNode = rightNode.left;
413
414         disconnectNodes(node, rightNode);
415         disconnectNodes(fatherNode, node);
416         disconnectNodes(rightNode, subNode);
417
418
419         connectNodes(rightNode, node);
420         connectNodes(node, subNode);
421         if (node != treeHead) // 如果 node 不是 treeHead, 那么就链接
422             connectNodes(fatherNode, rightNode);
423         else treeHead = rightNode; // 如果 node 是treeHead, 那么就链接给 rightNode
424     }
425
426     // 右旋转node
427     private void rightRotateNode(Nodes node) {
428         Nodes fatherNode = node.middle;
429         Nodes leftNode = node.left;
430         Nodes rightNode = node.right;
431
432
433         Nodes subNode = null;
434         if (leftNode != null) subNode = leftNode.right;
435
436         disconnectNodes(node, leftNode);
437         disconnectNodes(fatherNode, node);
438         disconnectNodes(leftNode, subNode);
439
440
441         connectNodes(leftNode, node);
442         connectNodes(node, subNode);
443         if (node != treeHead) // 如果 node 不是 treeHead, 那么就链接
444             connectNodes(fatherNode, leftNode);
445         else treeHead = leftNode; // 如果 node 是 treeHead, 那么就链接给 leftNode
446     }
447
448     // 如果新插入节点是非根节点, 而且父节点是红色, 叔叔节点是红色, 执行OptionA
449     // 传入的Node是插入节点

```

```

450 // 返回值如果是true代表仍然需要递归。false表示可以结束
451 private void insertOperationA(Nodes node) {
452     // 把父节点设置为黑色
453     node.middle.color = false;
454     // 把叔叔节点设置为黑色
455     getUncleNodes(node).color = false;
456     // 把爷爷设置为红色
457     node.middle.middle.color = true;
458     // 如果爷爷是根节点，就把爷爷设置为黑色
459     if (node.middle.middle == treeHead) node.middle.middle.color = false;
460     else // 如果爷爷不是根节点，就需要递归重新判断，判断对象是爷爷节点
461         rule(node.middle.middle);
462 }
463
464 // 获取叔叔节点
465 private Nodes getUncleNodes(Nodes node) {
466     if (node.middle.middle == null) {
467         System.out.println();
468     }
469     boolean position = getPosition(node.middle.middle, node.middle);
470     if (position) // 如果node的父节点是node的爷爷节点的右节点，那么uncle节点就
是.middle.middle.left
471         return node.middle.middle.left;
472     else // 如果node的父节点是node的爷爷节点的左节点，那么uncle节点就
是.middle.middle.right
473         return node.middle.middle.right;
474 }
475
476 // 取消这两个点的链接，第一个传入的是父亲节点（强制），第二个传入的是子节点
477 private void disconnectNodes(Nodes A, Nodes B) {
478     // 特殊情况
479     if (A != null && B != null && (A.key == B.key)) {
480         B.middle = null;
481         if (A.right == B) A.right = null;
482         else A.left = null;
483         return;
484     }
485     // 一般情况
486     if (A == null || B == null) return;
487     if (getPosition(A, B)) A.right = null;
488     else A.left = null;
489     B.middle = null;
490 }
491
492 // 链接这两个节点，第一个传入的是父亲节点（强制），第二个传入的是子节点
493 private void connectNodes(Nodes A, Nodes B) {
494     if (A == null || B == null) return;
495     if (getPosition(A, B)) A.right = B;
496     else A.left = B;
497     B.middle = A;
498 }
499
500 // 返回position信息，第一个传入的是父亲节点（强制），第二个传入的是子节点

```

```

501 // true 代表子节点在父节点右边
502 // false 代表子节点在父节点左边
503 private boolean getPosition(Nodes A, Nodes B) {
504     return B.key > A.key;
505 }
506
507 private void insertTreeHead(int key) {
508     treeHead = new Nodes();
509     treeHead.key = key;
510     treeHead.color = false; // 根节点必须颜色是黑色 (false) 的
511 }
512
513 // getColor 能处理Nil节点 (null节点), 返回false (黑色)
514 // 如果不是Nil节点, 就返回节点默认颜色
515 private boolean getColor(Nodes node) {
516     if (node == null) return false;
517     return node.color;
518 }
519
520 private static class Nodes {
521     int key;
522     Nodes left;
523     Nodes right;
524     Nodes middle;
525     boolean color = true; // true代表这个节点是红色
526 }
527 }

```

[6] 哈希表

简单哈希

包含增删查操作。是[LeetCode这题的答案](#)

查看这个视频初步了解[哈希表](#)。然后进一步了解[哈希表](#)。

```
1  class MyHashSet {
2      private static class Node{
3          int value;
4          Node next;
5          Node prev;
6          public Node(int value, Node next, Node prev) {
7              this.value = value;
8              this.next = next;
9              this.prev = prev;
10         }
11     }
12     private final Node[] hashtable;
13     private final int modNumber;
14     private int size;
15     public MyHashSet() {
16         int size = 200;
17         modNumber = modulo(size);
18         hashtable = new Node[size];
19     }
20
21     public void add(int key) {
22         int hash = hash(key);
23         if(hashtable[hash] == null) {
24             hashtable[hash] = new Node(key,null,null);
25             return;
26         }
27         Node search = hashtable[hash];
28         while(search.next != null)
29         {
30             if(search.value == key)
31                 return;
32             search = search.next;
33         }
34         if(search.value == key)
35             return;
36         search.next = new Node(key,null,search);
37     }
38
39     public void remove(int key) {
40         Node node = getIfContains(key);
41         if(node == null) return;
42         int hashKey = hash(key);
43         if(hashtable[hashKey].value == key)
44         {
```

```

45         hashtable[hashKey] = node.next;
46         if(hashtable[hashKey] != null) hashtable[hash(key)].prev = null;
47         return;
48     }
49
50     if(node.prev != null)
51         node.prev.next = node.next;
52     if(node.next != null)
53         node.next.prev = node.prev;
54 }
55
56 public boolean contains(int key) {
57     return getIfContains(key) != null;
58 }
59 private Node getIfContains(int key) {
60     Node search = hashtable[hash(key)];
61     if(search == null) return null;
62     while(search != null) {
63         if(search.value == key) return search;
64         search = search.next;
65     }
66     return null;
67 }
68 private int hash(int key) {
69     return key % modNumber;
70 }
71 private int modulo(int number){
72     for(int i=number; i>0; i--){
73         if(isPrime(i))
74             return i;
75     }
76     return size;
77 }
78 private boolean isPrime(int number) {
79     int sqrt = (int) Math.sqrt(number);
80     for(int i = 2; i <= sqrt; i++){
81         if(number % i == 0){
82             return false;
83         }
84     }
85     return true;
86 }
87 }

```

[7] 栈

后缀表达式

CPT102

```
1 public class InfixToPostfix {
2     public static void main(String[] args) {
3         System.out.println(compile("{3+[d-7*(g+5)]/w} "));
4         System.out.println(compile(" (a+n)*(b-8*m)"));
5         System.out.println(compile("b/v^7"));
6         System.out.println(compile("[ (4+b) -2)"));
7     }
8
9
10    public static String compile(String string) {
11        finalResult = new StringBuilder();
12        stack = new Stack<>();
13        string = deleteSpace(string); // 删除空格
14        string = fixBracket(string); // 把[、{、}、]这些括号转为()
15        if(!Balance.isBalanced(string))// 检测这个String是否括号匹配上了
16            System.err.println("Bracket is not balanced!");
17
18        for (int i = 0; i < string.length(); i++) {
19            char currentChar = string.charAt(i);
20            switch (currentChar) {
21                case '+', '-', '*', '/', '^' -> {
22                    charOperator(currentChar);
23                }
24                case '(', ')' -> {bracketOperator(currentChar);}
25                default -> finalResult.append(currentChar);
26            }
27        }
28        popAll();
29        return finalResult.toString();
30    }
31
32
33    private static StringBuilder finalResult = new StringBuilder();
34
35    private static Stack<Character> stack = new Stack<>();
36
37    private static String fixBracket(String string) {
38        StringBuilder result = new StringBuilder(string);
39        for (int i = 0; i < result.length(); i++) {
40            switch (result.charAt(i)) {
41                case '[', '{' -> result.setCharAt(i, '(');
42                case ']', '}' -> result.setCharAt(i, ')');
43            }
44        }
45        return result.toString();
46    }
47 }
```

```

46 }
47
48 private static void popAll() {
49     while (!stack.isEmpty()) {
50         finalResult.append(stack.pop());
51     }
52 }
53
54 private static void charOperator(char operator) {
55     if(stack.isEmpty()){ // 如果栈空，直接放入
56         stack.push(operator);
57         return;
58     }
59     // 栈不为空，一直弹出优先级比自己高的，也就是要找到 popOrNot == false 的
60     while(!stack.isEmpty() && popOrNot(operator, stack.peek()))
61     {
62         finalResult.append(stack.pop());
63     }
64     stack.push(operator); // 放入operator
65 }
66 /*如果 peek < currentChar （优先级） 无需弹出
67 如果 peek >= currentChar （优先级） 需要弹出
68
69 例如 peek = * < currentChar = ^ 无需弹出
70 例如 peek = * >= currentChar = + 需要弹出*/
71 private static boolean popOrNot(char currentChar, char peek) {
72     if(peek == '(') return false; // 特殊，顶级优先级
73     // 同级别弹出
74     if(currentChar == peek) return true; // 特殊，相等要弹出（包括同类型^）
75     if((currentChar=='*' || currentChar=='/') && (peek=='*' || peek=='/')) return true;
76     // 同类型乘除法弹出
77     if((currentChar=='+' || currentChar=='-') && (peek=='+' || peek=='-')) return true;
78     // 同类型加减法弹出
79
80     // 不同级别
81     if(currentChar == '^') return false; // current 是最高优先级的^，不弹出
82     if(peek == '^') return true; // peek是最高优先级的^，直接弹出
83
84     // 现在current和peek 只能是 +-* / 之一
85     if(currentChar == '*' || currentChar == '/') return false; // current是乘除法，无需
86     弹出
87     if((currentChar == '+' || currentChar=='-') && (peek=='*' || peek=='/')) return true;
88     // peek是乘除法，弹出
89
90     System.err.println("STACK ERROR. Current Char [" + currentChar + "] Compare Char [" + peek + "]");
91     return false;
92 }
93 // 处理括号
94 private static void bracketOperator(char currentChar) {
95     if(currentChar == '('){
96         stack.push(currentChar);

```

```
94         return;
95     }
96     // 剩下的是 ")"
97     while(!stack.isEmpty() && stack.peek() != '(')
98         finalResult.append(stack.pop());
99     if(!stack.isEmpty() && stack.peek() == '(')
100         stack.pop();
101
102 }
103 // 删除空格
104 private static String deleteSpace(String string) {
105     if (string == null || string.isEmpty()) return "";
106     StringBuilder finalResult = new StringBuilder(string);
107     for(int i = 0; i < finalResult.length(); i++) {
108         while (i < finalResult.length() && finalResult.charAt(i) == ' ')
109             finalResult.deleteCharAt(i);
110     }
111     return finalResult.toString();
112 }
113
114 }
```


[8] 动态规划DP

最长公共子序列 LCS

Longest Common Subsequence

INT102 PPT

请参考[力扣这一题](#)

1143. 最长公共子序列

已解答 

中等

 相关标签

 相关企业

 提示

Ax

给定两个字符串 `text1` 和 `text2`，返回这两个字符串的最长 **公共子序列** 的长度。如果不存在 **公共子序列**，返回 `0`。

一个字符串的 **子序列** 是指这样一个新的字符串：它是由原字符串在不改变字符的相对顺序的情况下删除某些字符（也可以不删除任何字符）后组成的新字符串。

- 例如，`"ace"` 是 `"abcde"` 的子序列，但 `"aec"` 不是 `"abcde"` 的子序列。

两个字符串的 **公共子序列** 是这两个字符串所共同拥有的子序列。

示例 1：

输入：text1 = "abcde", text2 = "ace"

输出：3

解释：最长公共子序列是 "ace"，它的长度为 3。

示例 2：

输入：text1 = "abc", text2 = "abc"

输出：3

解释：最长公共子序列是 "abc"，它的长度为 3。

示例 3：

输入：text1 = "abc", text2 = "def"

输出：0

解释：两个字符串没有公共子序列，返回 0。

状态转移方程：

$$F(x, y) = \begin{cases} F(x-1, y-1) + 1 & \text{if } \text{text}_1[x-1] = \text{text}_2[y-1] \\ \max(F(x-1, y), F(x, y-1)) & \text{if } \text{text}_1[x-1] \neq \text{text}_2[y-1] \end{cases}$$

		j	0	1	2	3	4	5	6
		i		b	d	c	a	b	a
0		0	0	0	0	0	0	0	0
1	a	0	0	0	0	1	1	1	1
2	b	0	1	1	1	1	2	2	2
3	c	0	1	1	2	2	2	2	2
4	b	0	1	1	2	2	3	3	3
5	d	0	1	2	2	2	3	3	3
6	a	0	1	2	2	3	3	4	4
7	b	0	1	2	2	3	4	4	4

```

1  class Solution {
2      public int longestCommonSubsequence(String text, String pattern) {
3          int[][] F = new int[pattern.length() + 1][text.length() + 1];
4          for(int x = 1; x <= pattern.length(); x++)
5              {
6                  for(int y = 1; y <= text.length(); y++){
7                      if(text.charAt(y-1) == pattern.charAt(x-1))
8                          F[x][y] = F[x-1][y-1] + 1;
9                      else
10                         F[x][y] = Math.max(F[x-1][y], F[x][y-1]);
11                 }
12             }
13         return F[pattern.length()][text.length()];
14     }
15 }

```

[9] 搜索与回溯

N-皇后问题

[Leetcode 题](#)，出现在 INT102 课件中，时间复杂度 $O(N!)$

51. N 皇后

已解答

困难 相关标签 相关企业 Aa

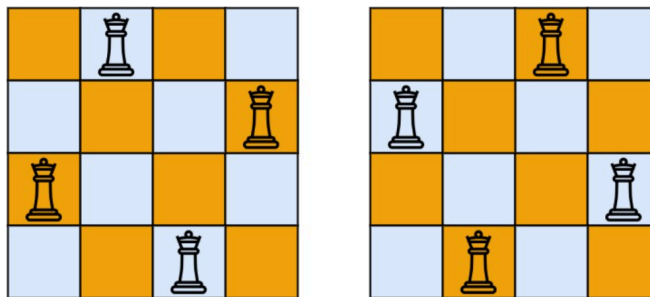
按照国际象棋的规则，皇后可以攻击与之处在同一行或同一列或同一斜线上的棋子。

n 皇后问题 研究的是如何将 **n** 个皇后放置在 $n \times n$ 的棋盘上，并且使皇后彼此之间不能相互攻击。

给你一个整数 **n**，返回所有不同的 **n 皇后问题** 的解决方案。

每一种解法包含一个不同的 **n 皇后问题** 的棋子放置方案，该方案中 'Q' 和 '.' 分别代表了皇后和空位。

示例 1:



输入: n = 4

输出: [[".Q..", "...Q", "Q...", "...Q."], [".Q..", "Q...", "...Q", ".Q.."]]

解释: 如上图所示，4 皇后问题存在两个不同的解法。

```
1 class Solution {
2     private final List<List<String>> result = new ArrayList<>();
3     private final List<Pair> currentPairs = new ArrayList<>();
4     private int SIZE;
5
6     private boolean validate(List<Pair> queens) {
7         if (queens.size() <= 1) return true;
8         int targetX = queens.getLast().x;
9         int targetY = queens.getLast().y;
10        for (int i = 0; i < queens.size() - 1; i++) {
11            if (queens.get(i).x == targetX || queens.get(i).y == targetY ||
12                Math.abs(targetX - queens.get(i).x) == Math.abs(targetY - queens.get(i).y))
13                return false;
14        }
15        return true;
16    }
17
18    public List<List<String>> solveNQueens(int n) {
19        SIZE = n;
20        execute(0,0);
21        return result;
22    }
23
24    // 添加答案
25    private void formTheAnswer() {
```

```

25     List<String> answer = new ArrayList<>();
26     String str = ".".repeat(SIZE);
27     for (int i = 0; i < SIZE; i++) {
28         answer.add(str);
29     }
30     for (Pair currentPair : currentPairs) {
31         StringBuilder sb = new StringBuilder(answer.get(currentPair.x));
32         sb.deleteCharAt(currentPair.y);
33         sb.insert(currentPair.y, 'Q');
34         answer.set(currentPair.x, sb.toString());
35     }
36     result.add(answer);
37 }
38
39 private boolean execute(int x, int y) {
40     if (x >= SIZE) return true;
41     if (y >= SIZE) return false;
42     currentPairs.add(new Pair(x, y));
43     if (!validate(currentPairs)) { // 如果当前位置(x,y)不满足, 就返回(x,y+1)的结果
44         currentPairs.removeLast();
45         return execute(x, y + 1);
46     } else { // 如果当前位置(x,y)满足:
47         if (!execute(x + 1, 0)) { // 如果其下一层(x+1,0)不满足, 那么就返回(x,y+1)
48             currentPairs.removeLast();
49             return execute(x, y + 1);
50         } else { // 如果其下一层(x+1,0)满足, 那么就保存结果, 继续下一位(x,y+1)
51             formTheAnswer();
52             currentPairs.removeLast();
53             return execute(x,y+1);
54         }
55     }
56 }
57
58 static class Pair {
59     int x;
60     int y;
61
62     public Pair(int x, int y) {
63         this.x = x;
64         this.y = y;
65     }
66 }
67 }

```